

Hypergraph Foundation Model

Yue Gao , Senior Member, IEEE, Yifan Feng , Shiquan Liu, Xiangmin Han , Member, IEEE, Shaoyi Du , Zongze Wu , Member, IEEE, and Han Hu , Member, IEEE

Abstract—Hypergraph neural networks (HGNNs) effectively model complex high-order relationships in domains like protein interactions and social networks by connecting multiple vertices through hyperedges, enhancing modeling capabilities, and reducing information loss. Developing foundation models for hypergraphs is challenging due to their distinct data, which includes both vertex features and intricate structural information. We present Hyper-FM, a Hypergraph Foundation Model for multi-domain knowledge extraction, featuring Hierarchical High-Order Neighbor Guided Vertex Knowledge Embedding for vertex feature representation and Hierarchical Multi-Hypergraph Guided Structural Knowledge Extraction for structural information. Additionally, we curate 11 text-attributed hypergraph datasets to advance research between HGNNs and LLMs. Experiments on these datasets show that Hyper-FM outperforms baseline methods by approximately 13.4%, validating our approach. Furthermore, we propose the first scaling law for hypergraph foundation models, demonstrating that increasing domain diversity significantly enhances performance, unlike merely augmenting vertex and hyperedge counts. This underscores the critical role of domain diversity in scaling hypergraph models.

Index Terms—Hypergraph neural networks, foundation model, high-order learning, hypergraph learning.

I. INTRODUCTION

HYPERGRAPH neural networks (HGNNs) have garnered significant attention due to their ability to perform high-order association and collaborative learning on data. They have been successfully applied across various domains, including protein interaction network analysis [1], [2], brain network analysis [3], [4], and social network analysis [5], [6]. Compared to traditional graphs, hypergraphs allow hyperedges to connect

more than two vertices, thereby offering stronger modeling capabilities and reducing information loss from real-world relational data. This enhanced structural representation grants HGNNs superior data representation and learning abilities.

Foundation models have been pivotal in advancing artificial intelligence, particularly in fields like computer vision (CV) [7], [8] and natural language processing (NLP) [9], [10]. In CV, models such as Vision Transformers [11] leverage large-scale datasets to learn rich visual representations, enabling superior performance across a wide range of tasks. Similarly, in NLP, models like BERT [12], [13] and GPT [14], [15] utilize extensive textual corpora to capture intricate language patterns, facilitating advancements in language understanding and generation. However, hypergraphs represent a distinct form of high-order relational structure, with data representations fundamentally different from images and text. The input to hypergraph models comprises two distinct types of information: vertex feature representations and hypergraph structural information. This dual nature poses substantial challenges in designing and implementing foundation models for hypergraphs. Constructing a unified hypergraph foundation model for multi-domain hypergraph data necessitates bridging the diverse vertex representations and hypergraph structures across different domains. This paper explores methods to extract unified hypergraph knowledge to generate a robust hypergraph foundation model, thereby enhancing performance in target hypergraph domains. We address these challenges from two perspectives: vertex knowledge extraction and hypergraph structural knowledge extraction.

Regarding the extraction of knowledge from vertex features, existing hypergraph models predominantly utilize Bag-of-Words (BoW) [16] representations. This approach leads to ever-increasing feature dimensions as textual descriptions grow, and varying vertex feature dimensions across different domains complicate the alignment of vertex features. Current methods attempt to extract fixed-dimensional embeddings from raw textual data using techniques like Singular Value Decomposition (SVD) [17] or Language Models (LM) [12], [13], [14], [15]. However, these approaches often overlook structural information, resulting in suboptimal feature representations. For instance, vertices with identical feature representations but located in different positions within the hypergraph should possess distinct semantic embeddings. Directly using LMs for feature extraction fails to incorporate structural information, raising the question: how can structural information be injected into LM-extracted features to enhance their discriminative power?

In terms of extracting structural knowledge from hypergraphs, the most straightforward approach is to pre-train [18], [19], [20]

Received 13 February 2025; revised 3 December 2025; accepted 4 December 2025. Date of publication 23 December 2025; date of current version 6 March 2026. This work was supported in part by Beijing Natural Science Foundation under Grant L242167, in part by the National Natural Science Foundation of China under Grant U24A20252, Grant 62327808, Grant U2336211, and Grant 623B2066, in part by the Fundamental Research Funds for the Central Universities under Grant xtr062025010, and in part by the Guangdong Major Project of Basic and Applied Basic Research under Grant 2023B0303000009. Recommended for acceptance by P. Zhao. (Corresponding authors: Zongze Wu; Han Hu.)

Yue Gao, Yifan Feng, and Xiangmin Han are with the School of Software, BN-Rist, THUICS, BLBCI, Tsinghua University, Beijing 100084, China (e-mail: kevin.gaoy@gmail.com; evanfeng97@gmail.com; simon.xmhan@gmail.com).

Shiquan Liu and Shaoyi Du are with the Institute of Artificial Intelligence and Robotics, College of Artificial Intelligence, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: quan3759@stu.xjtu.edu.cn; dushaoyi@xjtu.edu.cn).

Zongze Wu is with the College of Mechatronics and Control Engineering, Shenzhen University, Shenzhen 510006, China (e-mail: zzwu@szu.edu.cn).

Han Hu is with the Beijing Institute of Technology, Beijing 100811, China (e-mail: hhu@bit.edu.cn).

Digital Object Identifier 10.1109/TPAMI.2025.3647504

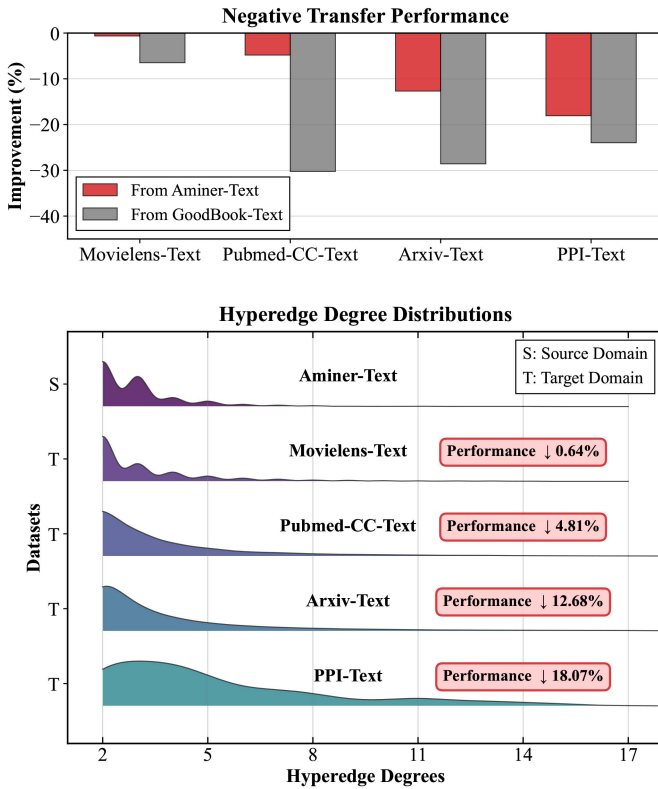


Fig. 1. Illustration of negative transfer phenomenon in hypergraph datasets.

independently on different domain-specific hypergraph structures to generate a hypergraph foundation model.

However, this approach yields poor results, as shown in Fig. 1. Pre-training on source domain hypergraph structures and then applying the model to target domain downstream tasks leads to negative transfer. This occurs because relational data from different domains exhibit significant structural pattern differences. Specifically, the lower part of Fig. 1 illustrates the underlying reason for performance degradation when transferring from Aminer-Text to other datasets. We plot the statistical distributions of hyperedge degrees and observe that Movielens-Text shares a distribution pattern similar to Aminer-Text, resulting in only a marginal performance drop of 0.64%. In contrast, as the distributional discrepancy increases, the performance degradation becomes more pronounced. For instance, the hypergraph distribution of PPI-Text differs substantially from that of Aminer-Text, with distinct peak positions and fluctuation patterns, leading to a significant performance drop of 18.07%. Consequently, how can multi-domain knowledge extraction and transfer be effectively achieved?

Existing methods typically concatenate structures directly, but we observe that this approach only performs adequately on small-scale datasets. It remains unclear whether large-scale knowledge extraction and transfer are feasible. Additionally, direct concatenation can result in the blending of domain-specific knowledge, as evidenced by the performance bottleneck observed in the experiments presented in Table VI. Thus, how can knowledge be transferred across domains while preserving the unique semantic information of each domain?

Moreover, research on hypergraph foundation models faces fundamental challenges at the data level. While there is an abundance of text-attributed graph data [21], [22], [23]—where each vertex is accompanied by textual descriptions—these descriptions facilitate better communication and collaboration between Graph Neural Networks (GNNs) and LM, spurring advancements in graph-based foundation models. In contrast, the text-attributed hypergraph (TAHG) dataset remains under-explored, representing a significant bottleneck for research at the intersection of hypergraphs and language models. The current lack of comprehensive TAHG datasets limits progress in this domain.

To address the aforementioned issues, we propose Hyper-FM: Hypergraph Foundation Model for Multi-Domain Hypergraph Knowledge Extraction. Specifically, for extracting vertex knowledge from hypergraph vertex features, we introduce Hierarchical High-Order Neighbor Guided Vertex Knowledge Embedding. This module hierarchically represents the hypergraph neighborhoods and defines a hierarchical domain prediction task to fine-tune the LM. This approach not only addresses the challenge of representing high-order domain information in large-scale hypergraphs but also injects domain information into the features extracted by the language model, achieving dual embedding of structural and semantic information. For extracting structural knowledge from hypergraphs, we present Hierarchical Multi-Hypergraph Guided Structural Knowledge Extraction. This method involves sampling sub-hypergraphs from different domain hypergraphs, clustering to construct a macro-hypergraph, and connecting different domain hypergraphs via bond vertices to form a Hierarchical Multi-Hypergraph. The sampling strategy effectively manages large-scale hypergraphs, while unsupervised clustering enhances collaborative learning among semantically similar vertices within the same domain. Additionally, the hierarchical structure with virtual vertices mitigates the direct impact of information from other domains, preventing the mixing of domain-specific information.

Furthermore, we have manually curated and constructed 11 TAHG datasets to facilitate cross-disciplinary research between HGNNs and LLMs. Experiments on these 11 hypergraph datasets demonstrate that our proposed foundation model outperforms baseline methods by an average of approximately 13.4%, validating its effectiveness. Additionally, we are the first to propose scaling laws for hypergraph computation. Our findings reveal that, unlike other domains, simply increasing the scale of data in terms of vertex and hyperedge counts does not enhance the foundational model's power, as illustrated in Fig. 5. However, increasing the number of domains—thereby exposing the model to a wider variety of relational structures—significantly enhances the model's performance. This suggests a promising direction for future development. In summary, our contributions are threefold:

- 1) We curate and release 11 text-attributed hypergraph (TAHG) datasets, fostering the integration of hypergraph computation and language model research.
- 2) We introduce the first hypergraph foundation model for extracting vertex and structural knowledge from multi-domain hypergraph data. Leveraging hierarchical

representations and a sampling-based Hierarchical Multi-Hypergraph, our model achieves an average performance improvement of approximately 13.4% across 11 TAHG datasets, demonstrating its effectiveness.

- 3) We propose the first scaling law for hypergraph foundation models, illustrating that increasing the number of vertices and hyperedges does not enhance model performance. Instead, augmenting the number of pre-training domains—thereby exposing the model to diverse relational structures—effectively improves the foundation model’s power.

II. RELATED WORKS

A. Foundation Models

Foundation models have profoundly transformed NLP and CV. In NLP, BERT [12] revolutionized context-aware embeddings through bidirectional training, boosting tasks like question answering. GPT-2 [14] and GPT-3 [15] further demonstrated the power of generative pre-training, with GPT-3’s 175B parameters enabling impressive few-shot learning. T5 [24] unified NLP tasks under a text-to-text framework, while RoBERTa [13] improved training efficiency by removing the next-sentence prediction objective and using larger datasets. In CV, models like DINO [25] leveraged self-supervised learning and knowledge distillation to learn rich visual representations without labels. SAM [26] advanced image segmentation, SegGPT [27] integrated segmentation and generation for complex tasks, and Visual ChatGPT [28] combined GPT-3 with visual understanding for interactive multimodal applications. Collectively, these advances underscore the broad impact of foundation models in modern machine learning.

B. Graph Foundation Models

In recent years, several innovative graph foundation models have emerged, significantly advancing the field of graph data processing. InstructGLM [29] is a generative language model specifically designed for graph structures, enabling users to create and modify graphs through natural language instructions, thereby enhancing interactivity with graph data. GraphGPT [30] combines graph neural networks with generative pre-trained transformers, focusing on generating and reasoning about graph data, which facilitates tasks such as graph description and generation through contextual information. LLaGA [31] aligns language models with graph structures to improve representation learning by jointly training on both data types, boosting performance on tasks like vertex classification and link prediction. Finally, GALLM [32] integrates graph information into language models, strengthening the connection between generated text and graph data, making it suitable for applications like knowledge graph construction and information retrieval.

C. Hypergraph Neural Networks

Hypergraph Neural Networks (HGNNs) have greatly advanced the modeling of high-order relationships in graph-structured data. HGNN [33] first introduced hypergraph

convolution to capture higher-order vertex relations, while HGNN⁺ [34] enhanced neighbor aggregation via multilevel information fusion. HyperGCN [35] implemented efficient hypergraph convolution layers, and HNHN [36] adopted hierarchical vertex representations for multilayer information propagation. HGAT [37] integrated self-attention with hypergraphs to dynamically learn vertex and hyperedge importance. UniGNN [38] provided a unified framework for diverse graph structures, improving model flexibility. HyperSAGE [39], inspired by GraphSAGE [40], introduced sampling-based aggregation for scalability. AllSet [41] leveraged set theory to optimize hypergraph representation, and DHHNN [42] is a variational autoencoder-based framework for multimodal data integration and vertex classification, which combines hyperbolic geometry, dynamic hypergraphs and self-attention to address the static structure limitation of existing models.

D. Pioneer in Hypergraph Foundation Models

Recent advances have seen pioneering hypergraph foundation models emerge, expanding hypergraph learning across domains. HGFM [43] is a high-order correlation-driven model for brain disease diagnosis, learning an encoder via self-supervised pre-training on high-order structures and adapting to downstream tasks through few-shot fine-tuning. HYPER [44] is a foundation model for inductive link prediction in knowledge hypergraphs, generalizing to novel entities and relations by encoding entities with their hyperedge positions. DMCL-HFM [45] targets whole slide images, using vertex-hyperedge dual-masked modeling and contrastive learning for pretraining to integrate local and global dependencies. The hypergraph-driven landmark detection model [46] supports cardiac function quantification, leveraging an adaptive dynamic hypergraph backbone and bidirectional hypergraph spatio-temporal decoder. Notably, these models are tailored to specific domain tasks (e.g., brain disease diagnosis), highlighting the lack of a universal hypergraph foundation model framework for multiple scenarios.

III. PRELIMINARIES

a) Text-attributed Hypergraphs: A hypergraph can be represented as $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V} = \{v_1, v_2, \dots\}$ is the set of vertices and $\mathcal{E} = \{e_1, e_2, \dots\}$ is the set of hyperedges. Typically, each vertex is associated with some textual descriptions, denoted as $\mathcal{T} = \{T_1, T_2, \dots\}$, where T_i is the textual description for vertex v_i .

b) Hypergraph Neural Networks and Task Definition: Hypergraph Neural Networks (HGNNs) are designed for feature representation learning on hypergraphs, facilitating feature convolution that is guided by the hypergraph structure. Classical HGNNs [33], [34] consist of two stages of message passing: message propagation from vertices to hyperedges and from hyperedges back to vertices, which can be defined as:

$$\mathbf{X}^{(l+1)} = \sigma \left(\mathbf{D}_v^{-\frac{1}{2}} \mathbf{H} \mathbf{D}_e^{-1} \mathbf{H}^\top \mathbf{D}_v^{-\frac{1}{2}} \mathbf{X}^{(l)} \boldsymbol{\Theta}^{(l)} \right), \quad (1)$$

where $\mathbf{D}_v$ and $\mathbf{D}_e$ are diagonal matrices representing the degree matrices of vertices and hyperedges, respectively. The matrix

$\mathbf{H} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{E}|}$ is the incidence matrix of the hypergraph, where $\mathbf{H}_{v,e} = 1$ indicates that vertex v is part of hyperedge e . $N = |\mathcal{V}|$ denotes the number of vertices. The vertex feature matrix is denoted by $\mathbf{X}^{(l)} \in \mathbb{R}^{N \times C^{(l)}}$, where $C^{(l)}$ represents the number of feature channels at layer l . The parameter $\Theta^{(l)} \in \mathbb{R}^{C^{(l)} \times C^{(l+1)}}$ consists of trainable weights for transforming features from layer l to layer $l + 1$. The activation function σ introduces non-linearity into the model.

The objective of this study is focused on the vertex classification task. Each vertex v is associated with a corresponding label Y_v , and the collection of all labels forms a label set $\mathcal{Y}$. The goal of the vertex classification task is to learn a mapping ϕ_θ from vertex embeddings to their respective labels, thereby enabling the classification of vertices with unknown labels based on their learned representations.

c) Hypergraph Foundation Models: In this study, we define Hypergraph Foundation Models to leverage multi-domain hypergraph data, specifically text-attributed hypergraphs, for obtaining pre-trained parameters of hypergraph neural networks through self-supervised learning. These pre-trained parameters serve as initialization for hypergraph neural networks applied to downstream tasks in other domains. By incorporating multi-domain hypergraph knowledge into the foundation model, we enhance the performance of hypergraph neural networks on target domain hypergraph data. The training process of the Hypergraph Foundation Model consists of two phases: pre-training with multi-domain hypergraph data and fine-tuning for specific domain data. The pre-training phase is formally defined as:

$$\theta^* = \arg \min_{\theta} \sum_{(\mathcal{T}_x, \mathcal{G}_x) \in \mathcal{D}^{\text{src}}} \mathcal{L}_{\text{pre}}(f(\mathcal{T}_x, \mathcal{G}_x | \theta)), \quad (2)$$

where θ^* denotes the optimized parameters of the Hypergraph Foundation Model after pre-training. The dataset $\mathcal{D}^{\text{src}} = \{(\mathcal{T}_1, \mathcal{G}_1), (\mathcal{T}_2, \mathcal{G}_2), \dots\}$ comprises multi-domain hypergraph data, with each pair $(\mathcal{T}_x, \mathcal{G}_x)$ containing the textual attributes $\mathcal{T}_x$ of the vertices and the structural information $\mathcal{G}_x$ of the hypergraph. The textual attributes $\mathcal{T}_x$ provide rich semantic information that complements the structural data captured by $\mathcal{G}_x$. The loss function $\mathcal{L}_{\text{pre}}$ corresponds to a predefined self-supervised hypergraph task designed to encode knowledge from multi-domain hypergraphs into the parameters. The function f_θ represents the hypergraph neural network model with trainable parameter θ . Through self-supervised tasks, the model captures and integrates knowledge from diverse hypergraph domains into the foundational parameters θ^* .

After pre-training, the foundation model is adapted to a specific target domain through fine-tuning. This process initializes the hypergraph neural network parameters with the pre-trained θ^* and optimizes them based on the target domain's data and labels. The fine-tuning process is defined as:

$$\begin{aligned} \theta' &= \arg \min_{\theta'} \sum_{(T_x, Y_x) \in (\mathcal{T}^{\text{tar}}, \mathcal{Y}^{\text{tar}})} \mathcal{L}_{\text{down}}(p_\theta(T_x, \mathcal{G}^{\text{tar}}), Y_x) \\ &\text{subject to } \theta'^{(0)} \leftarrow \theta^*, \end{aligned} \quad (3)$$

where $(\mathcal{T}^{\text{tar}}, \mathcal{G}^{\text{tar}}, \mathcal{Y}^{\text{tar}})$ represents the target domain hypergraph data, including textual vertex attributes $\mathcal{T}^{\text{tar}}$, hypergraph structure $\mathcal{G}^{\text{tar}}$, and vertex labels $\mathcal{Y}^{\text{tar}}$. The textual attributes $\mathcal{T}^{\text{tar}}$ provide semantic context for the vertices, while the structural information $\mathcal{G}^{\text{tar}}$ captures the relationships between vertices via hyperedges. The label set $\mathcal{Y}^{\text{tar}} = \{Y_1, Y_2, \dots, Y_N\}$ consists of labels for each vertex in the target hypergraph, essential for supervised fine-tuning. The loss function $\mathcal{L}_{\text{down}}$ pertains to the downstream vertex classification task, aiming to minimize the discrepancy between the predicted labels $p_\theta(T_x, \mathcal{G}^{\text{tar}})$ and the ground true labels Y^t . The parameter θ' represents the fine-tuned parameters of the hypergraph neural network tailored to the target domain. The predictive function $p_\theta(T_x, \mathcal{G}^{\text{tar}})$ maps the input textual attributes and hypergraph structures to label probabilities using the model parameters θ . The initialization condition $\theta'^{(0)} \leftarrow \theta^*$ ensures that fine-tuning begins with the pre-trained foundation parameters, facilitating the transfer of multi-domain knowledge to the specific task.

By fine-tuning the foundation model on downstream tasks, we transform a generic hypergraph neural network into a specialized model optimized for the target domain. This approach effectively combines prior knowledge from multiple domains with task-specific data and supervision, overcoming the performance limitations inherent to single-domain models and achieving maximal performance improvements. The fine-tuned parameters θ' retain the generalizable knowledge acquired during pre-training while adapting to the specificities of the target domain, resulting in superior classification performance. Through the proposed Hypergraph Foundation Model framework, we leverage multi-domain pre-training and domain-specific fine-tuning to develop robust and high-performing hypergraph neural networks across diverse application areas.

IV. METHODOLOGY

In this section, we present the Multi-Domain Hypergraph Foundation Model (Hyper-FM). First, we overview its design and core objectives. Hyper-FM has two core modules: Hierarchical High-Order Neighbor Guided Vertex Knowledge Embedding (generates structure-aware vertex embeddings) and Hierarchical Multi-Hypergraph Guided Structural Knowledge Extraction (integrates cross-domain structural knowledge). We then describe its self-supervised multi-domain pre-training and domain-specific fine-tuning, and compare it with classic graph foundation models to show its advantages.

A. Framework Overview

Fig. 2 presents our Multi-Domain Hypergraph Foundation Model. Given multi-domain text-attributed hypergraphs, we first utilize the Vertex Knowledge Embedding module to extract structure-aware vertex features by constructing hierarchical neighbor labels, thereby integrating high-order structural information into the vertex representations. Next, the Structural Knowledge Extraction module constructs a hierarchical

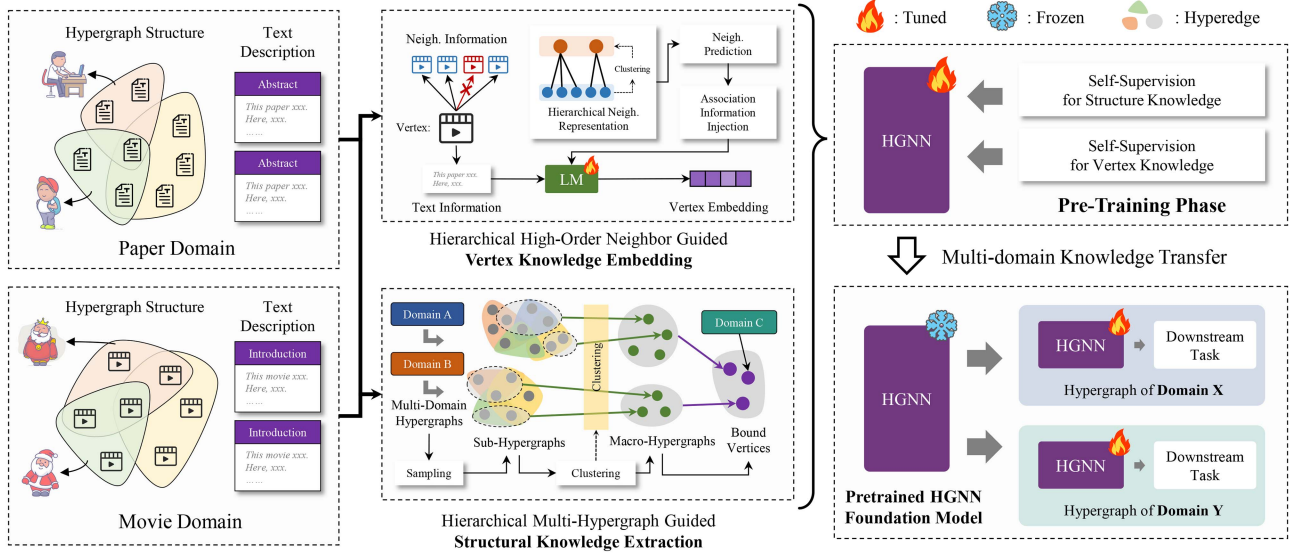


Fig. 2. Pipeline of the proposed Hypergraph Foundation Model (Hyper-FM).

multi-domain hypergraph by adding bond vertices that connect hypergraphs from different domains, facilitating cross-domain information propagation. During pre-training, we randomly sample structures from the multi-domain hypergraphs and use the structure-aware embeddings as input features to train the hypergraph foundation model in a self-supervised manner. For downstream tasks, the pre-trained model parameters are initialized for the specific domain's hypergraph, allowing the model to leverage the acquired multi-domain knowledge and improve performance on the target domain hypergraph data.

B. Hierarchical High-Order Neighbor Guided Vertex Knowledge Embedding

In this subsection, we introduce the Vertex Knowledge Embedding module, as illustrated in Fig. 3. Initially, vertex textual descriptions are encoded using a Language Model (LM) to capture semantic information. However, this approach alone does not fully leverage the connectivity information inherent in hypergraph data. To address scenarios where a vertex's textual description is semantically ambiguous, we enhance its representation by incorporating descriptions from neighboring vertices. To facilitate this, we design a self-supervised neighborhood prediction task that guides the LM to extract features that consider vertex associations within the hypergraph. Direct pairwise link prediction is computationally inefficient, and predicting all neighbor-pairs would result in an excessively large and unmanageable label space. Therefore, we propose a hierarchical high-order neighborhood representation method that employs hierarchical label prediction. This approach enables the LM to discern and integrate high-order relational structures within the hypergraph, thereby enriching the vertex embeddings with comprehensive structural and semantic knowledge.

1) *High-Order Neighborhood Vectorization*: We begin by defining the neighborhood vector within a hypergraph. For a

given vertex v , we first identify its incident hyperedges $\mathcal{N}_e(v)$. Subsequently, we collect all vertices u connected by these hyperedges, forming a vertex set. This set is then vectorized into a binary vector $\mathcal{D}_{\text{nbr}}(v) \in \{0, 1\}^{1 \times N}$, where N represents the total number of vertices in the hypergraph. The vectorization process is formalized as:

$$\mathcal{D}_{\text{nbr}}(v) = \text{vec}(\{u \mid u \in e, e \in \mathcal{N}_e(v), u \in \mathcal{V}\}), \quad (4)$$

where $\mathcal{D}_{\text{nbr}}(v)_{[i]} = 1$ if vertex v is connected to vertex i via a hyperedge, and 0 otherwise. Importantly, if two vertices v_i and v_j share identical neighborhoods, their vectors satisfy $\mathcal{D}_{\text{nbr}}(v_i) = \mathcal{D}_{\text{nbr}}(v_j)$. Leveraging this vectorization, we define the training objective for the LM as follows:

$$(\mathcal{D}_{\text{nbr}}(v_i) \simeq \mathcal{D}_{\text{nbr}}(v_j)) \propto \text{Similarity}(\phi(T_i), \phi(T_j)), \quad (5)$$

where ϕ denotes the LM, and T_i and T_j are the textual descriptions of vertices v_i and v_j , respectively. This objective ensures that vertices with similar neighborhood structures obtain comparable feature representations through the LM. Consequently, the LM is guided to incorporate high-order structural information from the hypergraph into the vertex embeddings, thereby enhancing both the semantic and structural integrity of the representations.

2) *Hierarchical Neighborhood Representation*: We transform neighborhood prediction into a multi-label classification problem. Specifically, for a vertex v , we aim to predict its connections to each vertex in the hypergraph using a binary vector of length N , where N is the total number of vertices. Here, the number of classification labels is $L = |\mathcal{V}|$. However, when dealing with hypergraphs containing a large number of vertices, the resulting vectors become excessively long, rendering the model difficult to train efficiently. Inspired by Extreme Multi-label Classification techniques [47], [48], [49], we address this challenge by adopting a hierarchical representation of neighbor relationships through clustering, grouping similar neighborhoods into unified clusters, as shown in Fig. 3.

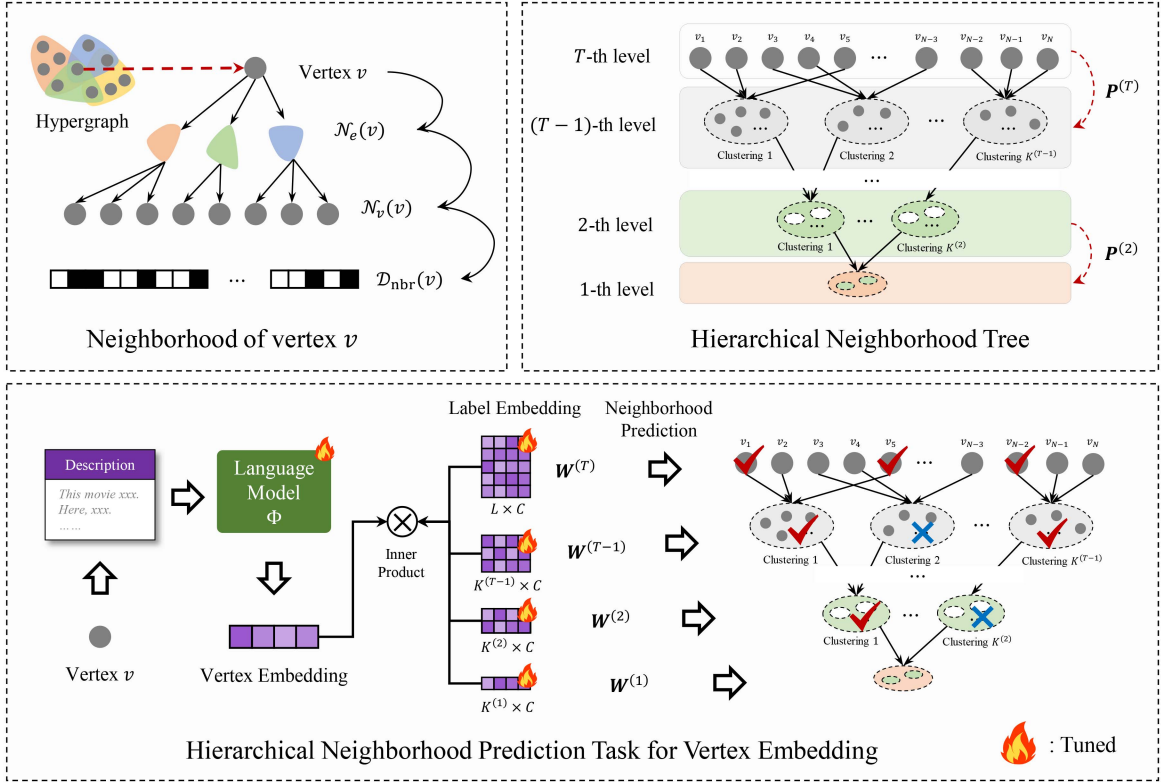


Fig. 3. Illustration of the hierarchical high-order neighbor guided vertex knowledge embedding module.

First, we define the fundamental feature representation for each label (i.e., the connection relationships of vertices) to facilitate clustering. For each label x , we aggregate the textual features of its associated vertices and normalize the resulting vector as follows:

$$\mathbf{Z}_x = \frac{\mathbf{v}_x}{\|\mathbf{v}_x\|}, \text{ where } \mathbf{v}_x = \sum_{u \in \mathcal{N}_v(x)} \psi(T_u), \forall x \in [L], \quad (6)$$

where ψ represents the basic text feature extraction network, such as TF-IDF [50]. It should be noted that ψ only extracts initial textual features, which are not directly fed into downstream neural networks but rather serve as the basis for clustering to provide references for hierarchical domains. T_u is the textual description of vertex u . The set $\mathcal{N}_v(x)$ comprises vertices associated with label x . The matrix $\mathbf{Z} \in \mathbb{R}^{L \times C}$ serves as the initial representation matrix for the labels.

To manage large-scale hypergraphs where the number of vertices (and thus labels) L can be both sparse and extensive, we employ a hierarchical clustering approach. At each clustering layer t , we project the label representations from the current level to the next coarser level using k -means clustering:

$$\mathbf{P}^{(t)} \leftarrow k\text{-means-clustering}(\mathbf{Z}^{(t)}, K^{(t-1)}), \quad (7)$$

where $\mathbf{P}^{(t)} \in \{0, 1\}^{K^{(t)} \times K^{(t-1)}}$ is the projection matrix mapping clusters from layer t to layer $t-1$, and $K^{(t)}$ denotes the number of clusters at layer t . The hierarchical process is repeated until the desired tree height T is achieved, with the finest granularity at layer T where $K^{(T)} = L$ and $\mathbf{Z}^{(T)} = \mathbf{Z}$. For

coarser layers, the label representations are updated as follows:

$$\mathbf{Z}^{(t-1)} = \mathbf{Z}^{(t)} \mathbf{P}^{(t)}, \quad (8)$$

where $\mathbf{Z}^{(t-1)} \in \mathbb{R}^{K^{(t-1)} \times C}$ represents the aggregated label features at the $(t-1)$ -th layer. Each row in $\mathbf{Z}^{(t-1)}$ corresponds to a group of vertices from the previous layer, effectively capturing higher-order neighborhood structures.

Given that vertices with similar neighborhoods are likely to share cluster memberships, this hierarchical clustering approach not only reduces the dimensionality of the label space but also enhances training efficiency by grouping related labels. Furthermore, we define the neighborhood label matrix at each layer t as:

$$\mathbf{Y}_{v,x}^{(t)} = \begin{cases} 1 & \text{if vertex } v \text{ is connected to label } x, \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

At the finest layer T , each label x corresponds to a specific vertex. In higher layers, each label represents a cluster of vertices. To propagate label information to coarser layers, we update the neighborhood label matrix as follows:

$$\mathbf{Y}^{(t-1)} = \text{bin}(\mathbf{Y}^{(t)} \mathbf{P}^{(t)}), \quad (10)$$

where $\mathbf{Y}^{(t)} \in \{0, 1\}^{N \times K^{(t)}}$, and the bin function binarizes the resulting matrix by setting all non-zero entries to 1. This hierarchical approach aggregates similar neighborhoods, creating a multi-level neighborhood representation. Consequently, structural information is progressively injected into the LM during

training, enabling the handling of larger-scale hypergraphs and improving both encoding and computational efficiency.

3) *Association Information Injection*: As illustrated in Fig. 3, the hierarchical structuring of neighborhood within the hypergraph facilitates the effective utilization of neighborhood similarities and addresses the challenges associated with representing a large number of vertices. In this section, we describe the method for injecting association information from the hypergraph into the LM to generate structure-aware vertex embeddings. To fully exploit the cross-level associations in the hierarchical neighborhood representation, we adopt a hierarchical progressive learning approach.

Specifically, we begin at the first level and iteratively predict each vertex's associations with other vertices or vertex clusterings. At each level t , we first obtain the predicted associations for the $(t-1)$ -th level, denoted as $\mathbf{Y}_{\text{pred}}^{(t-1)}$. To enhance training efficiency and robustness, inspired by beam search, we select the top k most significant association labels for each vertex:

$$\mathbf{Y}_{\text{pred}}^{(t-1)} = \text{Top} \left(\mathbf{W}^{(t-1)\top} \Phi(\mathcal{T}, \Theta), k \right), \quad (11)$$

where $\mathbf{Y}_{\text{pred}}^{(t-1)} \in \{0, 1\}^{N \times K^{(t-1)}}$ indicates whether each vertex is associated with the top k labels at level $t-1$. However, utilizing the entire vector for training would lead to an imbalance between positive and negative samples, resulting in suboptimal model performance. To mitigate this issue, we introduce a mask that selects a subset of samples for training at each level:

$$\mathbf{S}^{(t)} = \text{bin}(\mathbf{Y}_{\text{pred}}^{(t-1)} \mathbf{P}^{(t)\top} + \mathbf{Y}^{(t-1)} \mathbf{P}^{(t)\top}), \quad (12)$$

where $\mathbf{S}^{(t)} \in \{0, 1\}^{N \times K^{(t)}}$ is the sample selection matrix. The first term, $\mathbf{Y}_{\text{pred}}^{(t-1)} \mathbf{P}^{(t)\top}$, represents the associations predicted from the previous level, while the second term, $\mathbf{Y}^{(t-1)} \mathbf{P}^{(t)\top}$, denotes the true associations at the current level. The bin function converts the resulting matrix into a binary form, setting all non-zero entries to 1 and others to 0.

Based on this selection matrix, we define the optimization objective for learning the domain representations at level t as follows:

$$\min_{\mathbf{W}^{(t)}, \Theta} \sum_{i=1}^N \sum_{x: \mathbf{S}_{i,x}^{(t)} \neq 0} \mathcal{L}_{\text{BCE}} \left(\mathbf{Y}_{i,x}^{(t)}, \mathbf{W}_x^{(t)\top} \Phi(\mathcal{T}, \Theta) \right), \quad (13)$$

where $\mathbf{W}^{(t)}$ and Θ denote the embeddings of cluster labels at level t and the parameters of the LM, respectively. We employ the Binary Cross-Entropy (BCE) loss function to compute the loss for each vertex-label association, facilitating the learning of connections between vertices and their corresponding domain clusters. This process is iteratively repeated from the first level to the final level T until convergence, resulting in a LM that is aware of the hypergraph's structural associations. Consequently, the extracted vertex embeddings encapsulate both the semantic information from their textual descriptions and the structural knowledge from the hypergraph. Finally, the structure-aware vertex embeddings $\mathbf{X} \in \mathbb{R}^{N \times C}$ are generated as follows:

$$\mathbf{X} = \Phi(\mathcal{T}, \Theta^*), \quad (14)$$

where Θ^* represents the trained parameters of the LM, and $\mathcal{T}$ includes the textual descriptions of the vertices within the hypergraph. These vertex embeddings are subsequently utilized for constructing hierarchical multi-domain hypergraphs and pre-training the multi-domain foundation model.

C. Hierarchical Multi-Hypergraph Guided Structural Knowledge Extraction

Different domains exhibit distinct structural association patterns within their respective hypergraphs. Preliminary experiments (as shown in Fig. 1) indicate that isolated pretraining for each domain struggles to bridge the distributional discrepancies inherent in multi-domain hypergraph data. To overcome this limitation, we construct a hierarchical multi-domain hypergraph that integrates structural knowledge across domains, as shown in Fig. 4, thereby dismantling the barriers imposed by domain-specific structural constraints.

1) *Multi-Domain Hypergraphs Sampling*: Given hypergraph data from m distinct domains, we employ an independent sampling strategy for each domain-specific hypergraph to extract representative substructures. Specifically, for each domain $a \in \{1, 2, \dots, m\}$ with its corresponding hypergraph $\mathcal{G}^a = \{\mathcal{V}^a, \mathcal{E}^a\}$, we perform the following sampling process to obtain the sub-hypergraphs $\mathcal{G}_{\text{sub}}^a$:

$$\{\mathcal{G}_{\text{sub}}^1, \mathcal{G}_{\text{sub}}^2, \dots, \mathcal{G}_{\text{sub}}^m\} \xleftarrow{\text{sampling}} \{\mathcal{G}^1, \mathcal{G}^2, \dots, \mathcal{G}^m\}. \quad (15)$$

For a specific domain a , the sub-hypergraph $\mathcal{G}_{\text{sub}}^a = \{\mathcal{V}_{\text{sub}}^a, \mathcal{E}_{\text{sub}}^a\}$ is constructed as follows:

$$\begin{aligned} \mathcal{G}_{\text{sub}}^a &= \{\mathcal{V}_{\text{sub}}^a, \mathcal{E}_{\text{sub}}^a\} \quad \text{where} \\ \mathcal{V}_{\text{sub}}^a &= \{\text{BFS}(v) \mid v = \text{random_select}(\mathcal{V}^a)\}, \\ \mathcal{E}_{\text{sub}}^a &= \{\text{filter}(e, \mathcal{V}_{\text{sub}}^a) \mid e \in \mathcal{E}^a\}. \end{aligned} \quad (16)$$

where $\mathcal{V}_{\text{sub}}^a$ and $\mathcal{E}_{\text{sub}}^a$ denote the sampled vertex and hyperedge sets, respectively. The sampling process begins by randomly selecting a vertex v from the original vertex set $\mathcal{V}^a$ of domain d . Starting from this vertex, a breadth-first search (BFS) is conducted to expand the vertex set, adding neighboring vertices iteratively until a specified number of vertices is reached. This ensures that the sampled vertex set $\mathcal{V}_{\text{sub}}^a$ captures the local structural characteristics of the original hypergraph. Subsequently, the hyperedge set $\mathcal{E}_{\text{sub}}^a$ is derived by filtering the original hyperedges $\mathcal{E}^a$ to retain only those hyperedges where all constituent vertices are present in $\mathcal{V}_{\text{sub}}^a$. Formally, for each hyperedge $e \in \mathcal{E}^a$, it is included in $\mathcal{E}_{\text{sub}}^a$ if and only if $e \subseteq \mathcal{V}_{\text{sub}}^a$. This filtering step ensures that the sub-hypergraph $\mathcal{G}_{\text{sub}}^a$ maintains meaningful high-order relationships inherent in the original hypergraph while reducing complexity. This multi-domain sampling approach effectively captures the nuanced structural patterns of each domain's hypergraph, facilitating the subsequent clustering and integration steps.

2) *Multi-Domain Hypergraphs Clustering*: For each sampled sub-hypergraph structure $\mathcal{G}_{\text{sub}}^a$, we extract the feature matrix $\mathbf{X}_{\text{sub}}^a$ corresponding to the subset of vertices based on embeddings $\mathbf{X}^a$ from (14). We then apply k -means clustering to the embeddings to partition the vertices into k distinct clusters as

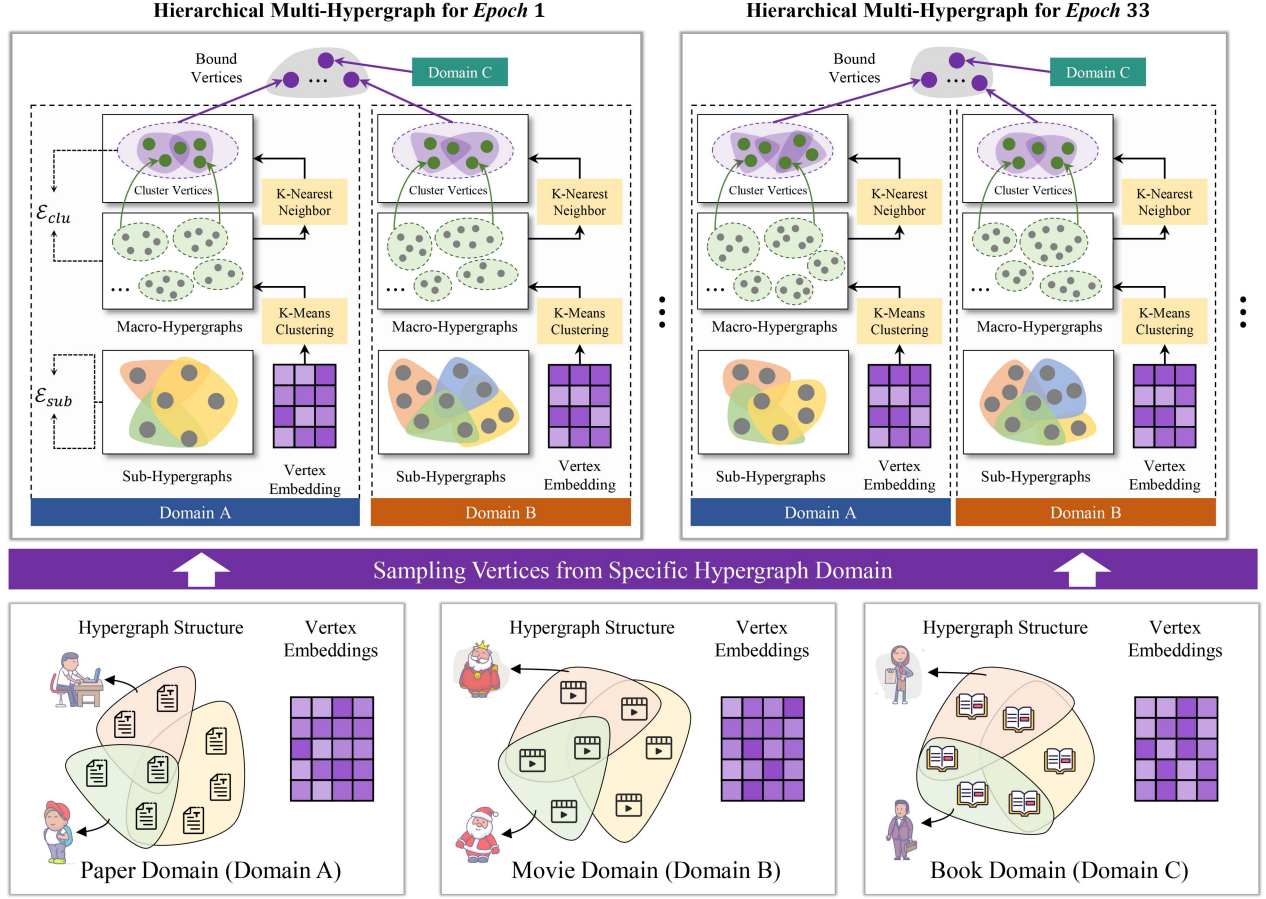


Fig. 4. Illustration of sampling and building Hierarchical Multi-Hypergraphs for the pretraining of hypergraph foundation model.

follows:

$$\{c_1^a, c_2^a, \dots, c_k^a\} \leftarrow k\text{-means-clustering}(\mathbf{X}_{\text{sub}}^a, k), \quad (17)$$

where k is a hyperparameter that determines the number of clusters, and each c_j^a represents a cluster containing a subset of vertices from domain a . It is important to note that each cluster c_j^a is a collection of vertices.

Subsequently, for each cluster c_j^a , we construct a virtual vertex to represent the cluster. The feature vector of this virtual cluster vertex, denoted as $\mathbf{x}_{a,c_j}^{\text{clu}}$, is obtained by aggregating the feature vectors of all vertices within the cluster:

$$\mathbf{x}_{a,c_j}^{\text{clu}} = \frac{1}{|c_j^a|} \sum_{u \in c_j^a} \mathbf{x}_u. \quad (18)$$

This aggregation ensures that the virtual vertex encapsulates the collective characteristics of the vertices within its cluster, providing a meaningful representation for higher-level structural analysis.

To capture high-order associations between these cluster vertices, we employ a k -nearest neighbors (k -NN) approach based on the aggregated cluster features $\mathbf{X}_{\text{clu}}^a$. This process constructs connections between clusters that exhibit similar feature representations, thereby encoding the structural relationships at a higher abstraction level. The hyperedges are then formed by combining these k -NN connections with the original hyperedges

derived from the clustering process:

$$\begin{cases} \mathcal{V}_{\text{clu}}^a = \{v_{a,1}^{\text{clu}}, v_{a,2}^{\text{clu}}, \dots, v_{a,k}^{\text{clu}}\}, \\ \mathcal{E}_{\text{clu}}^a = \text{knn}(\mathbf{X}_{\text{clu}}^a, k_c) \cup \{c_1^a, \dots, c_k^a\} \end{cases}, \quad (19)$$

where $\mathcal{V}_{\text{clu}}^a$ denotes the set of virtual cluster vertices for domain a , and $\mathcal{E}_{\text{clu}}^a$ represents the set of high-level hyperedges. The function $\text{knn}(\mathbf{X}_{\text{clu}}^a, k_c)$ generates hyperedges based on the k_c nearest neighbors in the cluster feature space, while $\{c_1^a, \dots, c_k^a\}$ includes the original clustered hyperedges. This clustering strategy effectively reduces the complexity of the hypergraph by grouping similar vertices, enabling the extraction of higher-order structural patterns within each domain. By constructing virtual vertices and establishing high-order associations, we enhance the representation of multi-domain hypergraphs, facilitating more efficient training and improved structural knowledge extraction in subsequent model stages.

3) *Hierarchical Multi-Hypergraph Construction*: In the subsequent step, we integrate the sampled sub-hypergraphs and their corresponding clusters from multiple domains to form the final hierarchical multi-domain hypergraph. Specifically, for each domain, a bond vertex is constructed and connected to its respective cluster vertices, thereby generating domain-specific hyperedges. Additionally, bond vertices across different domains are fully interconnected to facilitate the transfer of structural information between domains. The construction process is formalized

as follows:

$$\begin{cases} \mathcal{V}_{\text{bond}} = \{v_a^{\text{bond}}, v_b^{\text{bond}}, \dots, v_m^{\text{bond}}\} \\ \mathcal{E}_{\text{bond}} = \{(v_a^{\text{bond}}, v_b^{\text{bond}}, \dots, v_m^{\text{bond}}), \\ (v_a^{\text{bond}}, v_{a,1}^{\text{clu}}, v_{a,2}^{\text{clu}}, \dots, v_{a,k}^{\text{clu}}), \\ (v_b^{\text{bond}}, v_{b,1}^{\text{clu}}, v_{b,2}^{\text{clu}}, \dots, v_{b,k}^{\text{clu}}), \\ \dots \\ (v_m^{\text{bond}}, v_{m,1}^{\text{clu}}, v_{m,2}^{\text{clu}}, \dots, v_{m,k}^{\text{clu}})\} \end{cases} \quad (20)$$

where $\mathcal{V}_{\text{bond}}$ denotes the set of bond vertices, with each v_d^{bond} corresponding to domain $d \in \{a, b, \dots, m\}$. The set $\mathcal{E}_{\text{bond}}$ comprises two types of hyperedges:

- A hyperedge connecting all bond vertices across different domains, facilitating cross-domain information flow.
- Hyperedges that connect each bond vertex to its respective domain's cluster vertices, thereby consolidating the high-level structural information within each domain.

Next, for each bond vertex within a domain, we aggregate the features of its associated cluster vertices to form the bond vertex's feature vector:

$$\mathbf{x}_d^{\text{bond}} = \frac{1}{k} \sum_{j=1}^k \mathbf{x}_{d,c_j}^{\text{clu}}, \quad (21)$$

where $\mathbf{x}_d^{\text{bond}}$ represents the feature vector for bond vertex d , and $\mathbf{x}_{d,c_j}^{\text{clu}}$ denotes the feature vector of the j -th cluster vertex within domain d . This aggregation process ensures that each bond vertex encapsulates the aggregated structural features of its domain's clusters. Subsequently, we compile the features of all bond vertices across domains into a bond vertex feature matrix $\mathbf{X}_{\text{bond}} \in \mathbb{R}^{m \times C}$ by stacking the individual bond vertex features as follows:

$$\mathbf{X}_{\text{bond}} = \mathbf{x}_a^{\text{bond}} \parallel \mathbf{x}_b^{\text{bond}} \parallel \dots \parallel \mathbf{x}_m^{\text{bond}}. \quad (22)$$

Finally, we integrate all components to construct the hierarchical multi-domain hypergraph $\mathcal{G}_M$ as defined below:

$$\begin{cases} \mathcal{G}_M = \{\mathcal{V}_M, \mathcal{E}_M\}, \\ \mathcal{V}_M = \mathcal{V}_{\text{bond}} \cup \bigcup_{d=1}^m (\mathcal{V}_{\text{sub}}^d \cup \mathcal{V}_{\text{clu}}^d), \\ \mathcal{E}_M = \mathcal{E}_{\text{bond}} \cup \bigcup_{d=1}^m (\mathcal{E}_{\text{sub}}^d \cup \mathcal{E}_{\text{clu}}^d), \\ \mathbf{X}_M = \mathbf{X}_{\text{bond}} \parallel \mathbf{X}_{\text{sub}}^a \parallel \mathbf{X}_{\text{clu}}^a \parallel \dots \parallel \mathbf{X}_{\text{sub}}^m \parallel \mathbf{X}_{\text{clu}}^m \end{cases} \quad (23)$$

where $\mathcal{V}_M$ represents the unified set of vertices, including bond vertices, sampled sub-hypergraph vertices, and cluster vertices across all domains. $\mathcal{E}_M$ comprises all hyperedges, integrating bond hyperedges with sub-hypergraph and cluster hyperedges from each domain. $\mathbf{X}_M$ denotes the comprehensive feature matrix, concatenating the bond vertex features with those of sampled sub-hypergraph vertices and cluster vertices.

This hierarchical multi-domain hypergraph structure is derived through the sampling of multiple domain-specific hypergraphs, thereby supporting the training of a multi-domain foundation model. During training, sampling can be performed in each epoch to ensure comprehensive coverage of the hypergraph information. Notably, the virtual vertices within the hierarchical multi-domain hypergraph, comprising both cluster vertices and bond vertices, are characterized by aggregated features rather than trainable embeddings. This design choice enhances the transferability of the model, enabling the construction of

Algorithm 1: Toward Multi-Domain Hypergraph Pre-Training.

Input: Multi-domain text-attributed hypergraph datasets $\{(\mathcal{T}_1, \mathcal{G}_1), (\mathcal{T}_2, \mathcal{G}_2), \dots, (\mathcal{T}_m, \mathcal{G}_m)\}$, text description of vertices $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$, hypergraph neural networks with trainable parameters θ , LM $\{\Theta_j\}^j$ initialize raw neighborhood (label) embedding $\mathbf{Z}$ by (6), initialize neighbor embedding $\mathbf{W}$, neighborhood number at each level of hierarchical neighborhood representation $\mathcal{K} = \{K^{(1)}, K^{(2)}, \dots, K^{(t)}\}$, learning rate α .

Output: Pre-trained parameters θ^* of foundation model.

```

1: // Extract vertex features
2: for  $j$  in  $1, 2, 3, \dots, m$  do
3:   // Build hierarchical neighborhood representation
4:    $\{\mathbf{P}^{(t)}\}_{t=1}^T \leftarrow k\text{-means-clustering}(\mathbf{Z}, \mathcal{K})$ 
5:    $\{\mathbf{Y}^{(t)}\}_{t=1}^T \leftarrow \text{binary}(\mathbf{Y}^{(t)} \mathbf{P}^{(t)})$ 
6:   // Training for each neighborhood level
7:   for  $t$  in  $1, 2, 3, \dots, T$  do
8:      $\mathbf{Y}_{\text{pred}}^{(t-1)} \leftarrow \text{Top}(\mathbf{W}^{(t-1)\top} \Phi_j(\mathcal{T}_j, \Theta_j), k_t)$ 
9:      $\mathbf{S}^{(t)} = \text{binarize}(\mathbf{Y}_{\text{pred}}^{(t-1)} \mathbf{P}^{(t)\top} + \mathbf{Y}^{(t-1)} \mathbf{P}^{(t)\top})$ 
10:    update  $\mathbf{W}^t, \Theta_j$  by (13)
11:   end for
12:   // Generate vertex features for domain  $j$ 
13:    $\mathbf{X}_j \leftarrow \Phi_j(\mathcal{T}_j, \Theta_j)$ 
14: end for
15: Get multi-domain vertex feature set:  $\{\mathbf{X}_1, \dots, \mathbf{X}_m\}$ 
16: // Pretraining
17: while not converged do
18:   // Sample multi-domain hypergraphs
19:    $\{(\mathcal{G}_{\text{sub}}^j, \mathbf{X}_{\text{sub}}^j)\}_{j=1}^m \leftarrow \text{sampling}(\{(\mathcal{G}_j^j, \mathbf{X}_j)\}_{j=1}^m)$ 
20:   // Cluster vertices for each domain
21:   for  $j$  in  $1, 2, 3, \dots, m$  do
22:      $\{c_1^j, c_2^j, \dots, c_k^j\} \leftarrow k\text{-means-clustering}(\mathbf{X}_{\text{sub}}^j, k)$ 
23:      $(\mathcal{V}_{\text{clu}}^j, \mathcal{E}_{\text{clu}}^j) \leftarrow \text{by (19)}$ 
24:   end for
25:   // Build hierarchical multi-domain hypergraph
26:    $(\mathcal{G}_M, \mathbf{X}_M) \leftarrow \text{by (23)}$ 
27:   // Training HGNN for one epoch
28:    $\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}_{\text{pre}}(\mathcal{G}_M, \mathbf{X}_M)$ 
29: end while
30:  $\theta^* \leftarrow \theta$ 
31: return  $\theta^*$ 

```

arbitrary multi-domain hypergraphs and supporting effective learning across diverse domains.

D. Pretraining on Multi-Domain Hypergraphs

Here, we describe the pretraining procedure for obtaining the parameters of the hypergraph foundation model using multi-domain hypergraph data, as outlined in Algorithm 1.

Given a collection of m multi-domain text-attributed hypergraph datasets $\{(\mathcal{T}_1, \mathcal{G}_1), (\mathcal{T}_2, \mathcal{G}_2), \dots, (\mathcal{T}_m, \mathcal{G}_m)\}$, we begin by extracting structure-aware vertex features for each

domain-specific hypergraph. This extraction leverages the textual descriptions associated with each vertex, denoted by $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$, which are processed through domain-specific LM $\{\Theta_j\}_{j=1}^m$. During the pretraining process, each epoch begins with randomly sampling sub-hypergraphs from each domain to obtain $\{(\mathcal{G}_{\text{sub}}^j, \mathbf{X}_{\text{sub}}^j)\}_{j=1}^m$. For each domain j , the sampled vertex features $\mathbf{X}_{\text{sub}}^j$ undergo k -means clustering to partition the vertices into k distinct clusters, resulting in clusters $\{c_1^j, c_2^j, \dots, c_k^j\}$. Each cluster c_c^j comprises a set of vertices, and a corresponding virtual cluster vertex is created with its feature vector $\mathbf{x}_{j,c}^{\text{clu}}$ derived by averaging the features of all member vertices within the cluster. Subsequently, the hierarchical multi-domain hypergraph $\mathcal{G}_M$ is constructed by integrating the sampled sub-hypergraphs and their respective clusters using the methodology described in Section IV-B. The vertex feature matrix $\mathbf{X}_M$ is formed by concatenating the features of bond vertices, sampled sub-hypergraph vertices, and cluster vertices across all domains.

The constructed hierarchical multi-domain hypergraph $(\mathcal{G}_M, \mathbf{X}_M)$ is then input into the HGNN for training. The model parameters θ are updated by minimizing the pretraining loss $\mathcal{L}_{\text{pre}}$, which comprises both structural and feature-based self-supervised components as defined in (24):

$$\mathcal{L}_{\text{pre}} = \mathcal{L}_{\text{stru}}(\mathcal{G}_M, \tilde{\mathcal{G}}_M | \theta) + \mathcal{L}_{\text{feat}}(\mathbf{X}_M, \tilde{\mathbf{X}}_M | \theta), \quad (24)$$

where, the structural self-supervised loss $\mathcal{L}_{\text{stru}}$ is implemented using contrastive learning techniques applied to augmented structural views of the hypergraph, such as those employed in HyperGCL [18]. Following the design in SS-HT [51], we adopt both structural and feature-level self-supervised losses. Since the magnitudes of $\mathcal{L}_{\text{stru}}$ and $\mathcal{L}_{\text{feat}}$ are comparable, we simply aggregate them without additional weighting. This encourages the model to learn robust structural representations by distinguishing between different structural augmentations. The feature self-supervised loss $\mathcal{L}_{\text{feat}}$ involves masking portions of the vertex features and reconstructing them based on domain-specific information, similar to the approach used in GraphMAE [52]. This mechanism ensures that the model effectively captures both the structural dependencies and the semantic features of the vertices.

The pretraining process iterates until convergence, resulting in the optimized parameters θ^* of the hypergraph foundation model. These pre-trained parameters encapsulate comprehensive structural and feature information across multiple domains, thereby enhancing the model's capability to generalize and perform robustly on downstream hypergraph tasks.

E. Applying Knowledge to Downstream Hypergraphs

In this section, we elucidate the methodology for leveraging the pre-trained hypergraph foundation model on hypergraph data from new domains. Building upon the previously established pretraining steps, which utilized multi-domain hypergraphs to learn the foundational parameters θ of the HGNN, we demonstrate how these parameters can be effectively transferred to target domain hypergraphs to enhance their performance on specific tasks.

To apply the foundation model to a target domain, we begin by initializing the HGNN for the target hypergraph with the pre-trained parameters θ . This initialization imbues the target model with the structural and feature-based insights acquired from the diverse multi-domain hypergraphs during pretraining. Subsequently, the model architecture is adapted to the specific downstream task by appending an appropriate prediction head tailored to the task's requirements. For vertex classification tasks, an additional MLP layer is attached to each vertex's embedding. This MLP layer is responsible for mapping the enriched vertex embeddings to the corresponding label predictions, thereby facilitating accurate classification based on the learned representations. In the case of hypergraph classification tasks, a global pooling layer is employed to aggregate the vertex embeddings into a single hypergraph-level embedding. This aggregated representation is then passed through a classification layer to determine the hypergraph's class label, effectively capturing the overarching structural and semantic attributes of the hypergraph. For link prediction tasks, the approach involves directly utilizing the vertex embeddings to compute the probability of the existence of hyperedges. This is achieved by aggregating the embeddings of vertices that constitute a potential hyperedge and subsequently applying a suitable function to estimate the likelihood of their co-occurrence within a hyperedge. This strategy leverages the pre-trained embeddings to discern intricate relationships between vertices, thereby enhancing the model's predictive capabilities.

Throughout the fine-tuning process, the pre-trained parameters θ serve as a robust foundation, enabling the model to generalize effectively across various downstream tasks by incorporating prior knowledge from multiple domains. Empirical evaluations will subsequently demonstrate that the hypergraph foundation model not only encapsulates valuable prior information from multi-domain hypergraphs but also significantly elevates the performance benchmarks within target hypergraph domains for the vertex classification task. This transferability underscores the efficacy of the pretraining strategy in fostering versatile and high-performing hypergraph neural networks applicable to a wide range of real-world applications.

F. Discussions

We compare the proposed Hyper-FM with classical graph foundation models, focusing on core design differences and framework positioning, starting with Graph COordinators for PrEtraining (GCOPE) [53].

First, in vertex feature generation: GCOPE uses BoW [16] or TF-IDF [50] for feature extraction, then unifies dimensions via SVD [17], failing to fully leverage advanced language models (LMs) and ignoring inherent data relationships. Hyper-FM, by contrast, adopts LMs to extract fine-grained textual features, and fine-tunes LMs via hierarchical domain representations and domain prediction tasks, integrating structural information into features. Second, in multi-domain association structure generation: GCOPE concatenates multi-domain structures using virtual vertices connected to all domain vertices, causing information confusion among semantically distinct vertices. Hyper-FM

constructs hierarchical multi-domain hypergraphs via clustering, strengthening connections between semantically similar vertices and linking domain vertices to clustered ones, enabling orderly cross-domain knowledge transfer. Third, in scalability and transferability: GCOPE’s direct structure concatenation struggles with large-scale hypergraphs, and its domain-specific trainable virtual vertex embeddings require full retraining for new domains. Hyper-FM samples substructures per epoch to handle large-scale data, and generates virtual cluster/bond vertex features via aggregation, avoiding extra trainable parameters and supporting seamless new domain integration.

Beyond GCOPE, we further compare Hyper-FM with LLaGA [31] and GraphGPT [30], which represent an LLM-centered paradigm. LLaGA reorganizes graph nodes into structure-aware sequences and maps them to LLM token space via a projector, adapting graph data to LLM inputs with LLM as the core. GraphGPT integrates LLMs with graph knowledge through graph instruction tuning, including text-graph grounding and dual-stage tuning, enabling LLMs to understand graph structures. Both models rely on LLMs to capture graph information by “tokenizing” graphs. Hyper-FM, however, takes GNNs as the core framework; LMs only serve to align textual features across domains, assisting GNNs in learning better hypergraph knowledge—representing an essentially different design paradigm.

Overall, Hyper-FM outperforms GCOPE in vertex feature generation, multi-domain structure construction, and scalability. It also differs fundamentally from LLaGA/GraphGPT in framework positioning, setting a new benchmark for hypergraph foundation models.

V. EXPERIMENTS

In this section, we collect and build eleven text-attributed hypergraph datasets and conduct experiments on them to demonstrate the effectiveness of Hyper-FM.

A. Text-Attributed Hypergraph Datasets

This section introduces the text-attributed hypergraph datasets (TAHG) developed to support the Hypergraph Foundation Model. Recognizing that most existing hypergraph datasets feature vertex attributes primarily comprised of pre-extracted visual features or Bag-of-Words vector encodings, which are incompatible with large-scale models, we have manually constructed eleven text-attributed hypergraph datasets. These datasets are categorized into four distinct classes to encompass a wide range of application domains.

The *Citation Network Datasets* encompass Cora-CA-Text, Cora-CC-Text, Pubmed-CA-Text, Pubmed-CC-Text, Aminer-Text, Arxiv-Text, and OGBN-Arxiv. In these datasets, vertices represent papers with titles and abstracts serving as textual descriptions. For Cora and Pubmed, we collect textual descriptions based on the original paper IDs since this information is not available in current databases; Hyperedges are generated using co-author and co-citation relationships to capture the intricate interconnections within academic communities. For Arxiv-Text, we scrape data from the Arxiv repository, aggregating computer

science papers published from 2010 to 2023, selecting data vertices from five major categories (Computer Vision, Computation and Language, Robotics, Computers and Society, Cryptography and Security, Software Engineering), and forming hyperedges via co-citation relationships. Additionally, for Aminer-Text, we scrape data from the Aminer website and generate hyperedges based on co-author relationships. OGBN-Arxiv is an additional large-scale dataset directly adopted from public resources.¹ It is a directed citation network of Computer Science arXiv papers indexed by MAG [54], where each vertex denotes an arXiv paper with textual descriptions consisting of the original paper’s title and abstract. Each directed edge represents a citation relationship, and its task is 40-class subject area prediction (e.g., cs.AI, cs.LG).

The *Visual Media Datasets* include the MovieLens-Text dataset and IMDB-Text, where vertices correspond to movies enriched with textual descriptions, including the movie titles and introduction. For IMDB-Text, we scrape all movie data from the IMDB website for the years 2010 to 2023 and select data from five major film categories: Comedy, Horror, Documentary, Sci-Fi, and Animation. Hyperedges are generated based on co-director relationships to enable a detailed examination of film collaborations. For MovieLens-Text, we utilize the original data from the ml-latest dataset and construct hyperedges based on co-writer relationships among films, facilitating the exploration of narrative connections across different movies.

As for the *Book Domain*, we build the GoodBook-Text dataset, which models bibliographic information where vertices represent books accompanied by textual metadata. We utilize the original data from the GoodBook dataset and collect book names and details as textual descriptions for each vertex, allowing for a clear representation of books within the network.

As for the *Protein Domain*, we build the PPI-Text dataset, where vertices represent proteins and hyperedges denote protein-protein interactions. These hyperedges are supplemented with textual annotations that provide functional and structural insights, and the textual descriptions for each vertex consist of the protein sequences. This comprehensive collection of datasets facilitates the study of complex relationships across various domains, enhancing the applicability of our Hypergraph Foundation Model.

The statistics of these datasets are given in Table I. By incorporating rich textual attributes, these datasets enhance the capability of Hyper-FM to leverage semantic information across multiple domains, thereby facilitating more effective knowledge extraction and transfer learning in downstream tasks.

B. Experimental Settings

a) Compared Methods: In our experiments, we evaluate the performance of the proposed *Hyper-FM+Finetune* against a range of baseline methods to demonstrate its effectiveness. Firstly, we compare our approach with supervised methods that utilize the same labeled data. This includes a Multi-Layer Perceptron (MLP) with an identical number of layers to our

¹<https://ogb.stanford.edu/docs/nodeprop/>

TABLE I
STATISTICS RESULTS OF TEXT-ATTRIBUTED HYPERGRAPH DATASETS

Dataset	#Label	#Text	#Avg.	#V	#E
Aminer-Text	3	Title+Abstract	215.45	8,226	4,050
Cora-CA-Text	7	Title+Abstract	155.89	2,708	1,922
Cora-CC-Text	7	Title+Abstract	155.89	2,708	2,161
Pubmed-CA-Text	3	Title+Abstract	249.16	19,717	28,154
Pubmed-CC-Text	3	Title+Abstract	249.16	19,717	10,547
Arxiv-Text	5	Title+Abstract	194.10	22,886	15,478
Movielens-Text	5	Name+Intro.	43.26	18,479	8,271
IMDB-Text	5	Name+Intro.	57.43	34,619	13,290
GoodBook-Text	4	Name+Detail	182.53	5,834	7,920
PPI-Text	3	Sequence	587.84	401	381
OGBN-Arxiv	40	Title+Abstract	170.36	169,343	168,537

#V denotes “Number of vertices”, #E denotes “Number of hyperedges”, #Classes denotes “Number of classes”, #Text denotes “Type of text description”, #Avg. denotes “Average length of vertex text descriptions”. “Intro.” is short for “Introduction”.

foundation model, serving as a straightforward baseline that processes vertex features without leveraging hypergraph structures. Additionally, we benchmark against classical hypergraph neural networks such as HGNN [33], HGNN⁺ [34], HNHN [36], and UniGCN [38], which are well-established models in hypergraph representation learning. To assess the efficacy of our proposed multi-domain joint pretraining strategy, we also compare it with pretraining-based methods. Specifically, we employ Isolated Pretraining with Finetuning (IP+Finetune), which utilizes the same domain-specific hypergraph data for pretraining as our approach. However, IP+Finetune adopts a naive sequential pretraining strategy, where the model is pre-trained on one domain’s hypergraph data before proceeding to the next, without exploiting potential synergies between different domains during the pretraining phase.

Our proposed method, Hyper-FM+Finetune, embodies the Hypergraph Foundation Model strategy. For hypergraph pretraining, we utilize two self-supervised techniques: HyperGCL [18], a representative contrastive learning framework tailored for hypergraphs, and SS-HT [51], which leverages the reconstruction of hypergraph vertex features as a self-supervised objective. The backbone of our foundation model incorporates both HGNN and HGNN⁺ architectures. In the HyperGCL+HGNN configuration, we apply the HyperGCL self-supervised pretraining strategy to the HGNN architecture. Subsequently, the pre-trained HGNN parameters are transferred and fine-tuned on downstream hypergraph datasets from different domains. This joint pretraining approach allows the model to integrate multi-domain structural knowledge, thereby enhancing its generalization capabilities across diverse downstream tasks.

b) Implemental Details: We first conduct data preprocessing, including hyperedge construction, hypergraph structure optimization and vertex feature processing. When building hyperedges, any co-citation or co-director-like association forms a hyperedge; however, during dataset construction, we remove hyperedges with abnormally large degrees (> 40 , typically accounting for a small proportion) to reduce structural noise, and further eliminate isolated vertices from the remaining hypergraph to ensure valid node connectivity. For vertex features, we

use the bert-base-uncased [12] model to extract 768-dimensional textual features, then compress the dimension to 256 via Singular Value Decomposition (SVD) to lower computational complexity.

In the pretraining phase, we fix HGNN layers at 2, with a hidden dimension of 128 for both HGNN and HGNN⁺. The Vertex Knowledge Embedding module uses 4 hierarchical layers ($T = 4$, validated in [55]); the number of hierarchical label clusters K is calculated as $K = N/B$ ($B = 100$, N is sample count), following [47]—excessively small/large K degrades performance, especially small K leading to overcrowded clusters. While we adopt this fixed heuristic for consistency with prior work, we acknowledge that adaptive or dynamic determination of K within constrained ranges could further enhance model adaptability and performance, and we leave this as an important direction for future exploration. For “Multi-Domain Hypergraphs Sampling”, we adopt BFS: neighbor vertices are randomly sampled at 80% ~ 100%, expansion depth is constrained by total sampled vertices, and the sampling rate is dynamically adjusted to cover at least third-order neighbors of each vertex. After sampling 500 vertices per domain, we cluster them into 5 groups and generate hyperedges via second-order KNN. The Adam optimizer with a learning rate of 0.001 is used here.

In the fine-tuning phase, vertex classification serves as the downstream task, with a single-layer MLP as the classifier and only pre-trained HGNN layers fine-tuned to retain generalizable pretraining knowledge. For fair comparison, supervised baselines (MLP, HGNN, HGNN⁺, HNHN, UniGCN) have a hidden dimension of 128 (matching the pretraining backbone) and use Adam with a learning rate of 0.01 (consistent with the downstream task). Data splitting follows the C-way-1-shot setting [53]: training data is built under this setting, 100 vertices per class are used as the validation set, and the rest as the test set. For datasets with temporal information (e.g., OGBN-Arxiv), all samples were shuffled across publication years before random splitting, since chronological order is not the focus of our study and random sampling avoids potential bias. Performance is evaluated by the mean and variance of five independent runs to ensure reliability.

C. Results and Discussions

Tables II and III present the experimental results of our proposed Hyper-FM on eleven TAHG datasets. From these tables, we can draw the following four key observations.

Firstly, hypergraph neural network methods such as HGNN consistently outperform the MLP baseline across all datasets. This improvement underscores the effectiveness of leveraging hypergraph-structured associations, which facilitate the transmission of relational information and enable the model to capture more comprehensive features, thereby enhancing overall performance. Secondly, our proposed Hyper-FM method demonstrates a substantial performance enhancement of approximately 13.4% across all eleven datasets, with an exceptional improvement of 23.4% observed on the Pubmed-CC-Text dataset. Notably, we also observe a significant improvement of 14.3% on

TABLE II
EXPERIMENTAL RESULTS ON FIVE TEXT-ATTRIBUTED HYPERGRAPH DATASETS

	Methods	Aminer-Text	Cora-CA-Text	Cora-CC-Text	Pubmed-CA-Text	Pubmed-CC-Text
Supervised	MLP	0.3573 $\pm$.01	0.2399 $\pm$.03	0.3063 $\pm$.04	0.4123 $\pm$.02	0.3168 $\pm$.00
	HGNN	0.3916 $\pm$.01	0.2571 $\pm$.01	0.3570 $\pm$.04	0.4463 $\pm$.05	0.3248 $\pm$.09
	HGNN ⁺	0.3976 $\pm$.01	0.2575 $\pm$.04	0.3603 $\pm$.05	0.4397 $\pm$.03	0.3562 $\pm$.00
	HNHN	0.3813 $\pm$.02	0.2527 $\pm$.03	0.3525 $\pm$.05	0.4359 $\pm$.04	0.3574 $\pm$.08
	UniGCN	0.3845 $\pm$.01	0.2538 $\pm$.04	0.3530 $\pm$.05	0.4573 $\pm$.05	0.3705 $\pm$.06
IP + Finetune	HyperGCL + HGNN	0.3882 $\pm$.01	0.2415 $\pm$.05	0.3268 $\pm$.08	0.3749 $\pm$.01	0.3884 $\pm$.01
	HyperGCL + HGNN ⁺	0.3948 $\pm$.02	0.2436 $\pm$.05	0.3443 $\pm$.06	0.4010 $\pm$.01	0.3591 $\pm$.06
	SS-HT + HGNN	0.3901 $\pm$.02	0.2437 $\pm$.03	0.3322 $\pm$.07	0.4404 $\pm$.02	0.3848 $\pm$.00
	SS-HT + HGNN ⁺	0.3894 $\pm$.03	0.2583 $\pm$.04	0.3382 $\pm$.04	0.4539 $\pm$.03	0.3875 $\pm$.02
Hyper-FM + Finetune	HyperGCL + HGNN	0.4325$\pm$.02	0.2633 $\pm$.03	0.3706 $\pm$.04	0.4603 $\pm$.04	0.4009$\pm$.01
	HyperGCL + HGNN ⁺	0.4174 $\pm$.02	0.2654 $\pm$.03	0.3961$\pm$.04	0.4641 $\pm$.03	0.3972 $\pm$.01
	SS-HT + HGNN	0.3976 $\pm$.02	0.2777$\pm$.03	0.3525 $\pm$.05	0.4692 $\pm$.03	0.3942 $\pm$.00
	SS-HT + HGNN ⁺	0.3995 $\pm$.02	0.2707 $\pm$.06	0.3551 $\pm$.05	0.4708$\pm$.04	0.3948 $\pm$.00
IMP(%)		10.4%	8.0%	9.9%	7.1%	23.4%

The bold numbers indicate the best results in each column. The "IP" stands for Isolated Pretraining.

The "IMP" denotes the performance gain of the optimal method under the "Hyper-FM + Finetune" setting relative to its counterpart without pretraining.

TABLE III
EXPERIMENTAL RESULTS ON ANOTHER SIX TEXT-ATTRIBUTED HYPERGRAPH DATASETS

	Methods	Arxiv-Text	Movielens-Text	IMDB-Text	GoodBook-Text	PPI-Text	OGBN-Arxiv
Supervised	MLP	0.3345 $\pm$.02	0.2880 $\pm$.03	0.2146 $\pm$.09	0.2489 $\pm$.04	0.4194 $\pm$.09	0.2034 $\pm$.04
	HGNN	0.3523 $\pm$.05	0.3261 $\pm$.08	0.2533 $\pm$.03	0.3554 $\pm$.03	0.4306 $\pm$.08	0.2348 $\pm$.04
	HGNN ⁺	0.3825 $\pm$.08	0.3016 $\pm$.09	0.2205 $\pm$.09	0.3225 $\pm$.01	0.5065 $\pm$.08	0.2441 $\pm$.03
	HNHN	0.3753 $\pm$.09	0.2688 $\pm$.09	0.2528 $\pm$.07	0.3130 $\pm$.01	0.5261 $\pm$.05	0.2331 $\pm$.03
	UniGCN	0.3571 $\pm$.02	0.3269 $\pm$.07	0.2565 $\pm$.05	0.3607 $\pm$.04	0.4387 $\pm$.07	0.2559 $\pm$.04
IP + Finetune	HyperGCL + HGNN	0.3540 $\pm$.06	0.2384 $\pm$.02	0.2536 $\pm$.03	0.3278 $\pm$.03	0.4339 $\pm$.07	0.2657 $\pm$.04
	HyperGCL + HGNN ⁺	0.3206 $\pm$.04	0.2796 $\pm$.04	0.2586 $\pm$.04	0.3422 $\pm$.02	0.5084 $\pm$.06	0.2665 $\pm$.04
	SS-HT + HGNN	0.3505 $\pm$.04	0.3185 $\pm$.05	0.2486 $\pm$.08	0.3210 $\pm$.01	0.4355 $\pm$.04	0.2414 $\pm$.04
	SS-HT + HGNN ⁺	0.3783 $\pm$.07	0.3174 $\pm$.08	0.2466 $\pm$.08	0.3166 $\pm$.01	0.4742 $\pm$.03	0.2596 $\pm$.04
Hyper-FM + Finetune	HyperGCL + HGNN	0.4189 $\pm$.04	0.3464 $\pm$.08	0.2676 $\pm$.04	0.3588 $\pm$.04	0.5371 $\pm$.08	0.2764 $\pm$.01
	HyperGCL + HGNN ⁺	0.4198 $\pm$.07	0.3470$\pm$.06	0.2672 $\pm$.04	0.3703$\pm$.04	0.5742$\pm$.06	0.2789$\pm$.01
	SS-HT + HGNN	0.4044 $\pm$.05	0.3305 $\pm$.05	0.2786$\pm$.08	0.3589 $\pm$.04	0.5403 $\pm$.04	0.2759 $\pm$.04
	SS-HT + HGNN ⁺	0.4649$\pm$.05	0.3347 $\pm$.09	0.2775 $\pm$.08	0.3569 $\pm$.01	0.5387 $\pm$.03	0.2745 $\pm$.04
IMP(%)		21.5%	15.1%	9.9%	14.8%	13.4%	14.3%

the large-scale and complex vertex classification dataset OGBN-Arxiv, which demonstrates the effectiveness of Hyper-FM in handling large-scale and complex hypergraph data. This significant uplift highlights the efficacy of the Hypergraph Foundation Model. By conducting pretraining on multi-domain hypergraph datasets, Hyper-FM successfully integrates diverse domain-specific knowledge into the hypergraph, which in turn markedly boosts the performance of target domain models during downstream tasks. Thirdly, when comparing with pretraining-based methods, the IP+Finetune approach exhibits noticeably inferior performance relative to Hyper-FM+Finetune. In particular, the Cora-CC-Text dataset experiences negative transfer, resulting in decreased performance in the target domain. This decline can be attributed to the sequential nature of IP+Finetune's pretraining strategy, which despite utilizing self-supervised training to acquire multi-modal hypergraph domain knowledge, is susceptible to knowledge forgetting. Additionally, the significant disparities in knowledge distributions across different domains hinder the convergence of sequential pretraining, thereby adversely affecting performance on downstream tasks.

Lastly, within the Hypergraph Foundation Model framework, the choice of self-supervised strategy, whether HyperGCL or SS-HT, has a relatively minor impact on the overall results. In

contrast, the utilization of multi-domain hierarchical hypergraphs exerts a pronounced effect on performance. A similar trend is observed in the graph foundation model GCOPE [53], reinforcing the conclusion that the construction of robust association structures and the effective integration of cross-domain association information are critical focal points for advancing hypergraph foundation models. These observations collectively demonstrate that our Hyper-FM not only leverages the inherent advantages of hypergraph structures but also effectively integrates multi-domain knowledge through sophisticated pre-training strategies. This dual capability underpins the model's superior performance and highlights the importance of comprehensive structural and semantic knowledge integration in hypergraph-based learning frameworks.

D. More Evaluations and Analysis

In this subsection, we conduct ablation studies to evaluate the individual contributions of the *Vertex Knowledge Embedding* module and the *Structural Knowledge Extraction* module within our Hyper-FM on eleven TAHG datasets.

1) *Ablation Study on Vertex Knowledge Embedding*: Table IV presents the experimental results for various vertex feature

TABLE IV
ABLATION STUDY ON THE VERTEX KNOWLEDGE EXTRACTION

	BoW	LM	LM+NP
Pubmed-CA-Text	0.4260 $\pm$.03	0.4497 $\pm$.01	0.4603$\pm$.04
Movielens-Text	0.2602 $\pm$.02	0.2852 $\pm$.04	0.3464$\pm$.08
GoodBook-Text	0.3297 $\pm$.04	0.3142 $\pm$.02	0.3588$\pm$.04
PPI-Text	0.5032 $\pm$.07	0.4839 $\pm$.09	0.5371$\pm$.08
Aminer-Text	0.3940 $\pm$.01	0.4073 $\pm$.01	0.4325$\pm$.02
Cora-CA-Text	0.2101 $\pm$.02	0.2526 $\pm$.05	0.2633$\pm$.03
Cora-CC-Text	0.3059 $\pm$.04	0.3368 $\pm$.04	0.3706$\pm$.04
Pubmed-CC-Text	0.3977 $\pm$.01	0.3967 $\pm$.01	0.4009$\pm$.01
Arxiv-Text	0.2786 $\pm$.03	0.3231 $\pm$.07	0.4189$\pm$.04
IMDB-Text	0.2625 $\pm$.05	0.2637 $\pm$.04	0.2676$\pm$.04
OGBN-Arxiv	0.2667 $\pm$.02	0.2608 $\pm$.03	0.2789$\pm$.01

extraction methods. Specifically, we compare the performance of Bag-of-Words (BoW), direct feature extraction using the LM, and our proposed approach, LM with Neighborhood Prediction fine-tuning (LM+NP). The BoW method serves as a naive baseline by representing vertex features based on word frequency, while the LM approach leverages pre-trained language models to extract semantic features directly from the textual data. The LM+NP method enhances the LM by incorporating neighborhood information through a Neighborhood Prediction task during fine-tuning. From the results, it is evident that there is no significant difference in performance between the LM and BoW methods overall. Notably, the LM features demonstrate superior performance on the Pubmed-CA-Text, Movielens-Text, Aminer-Text, Cora-CA-Text, and Cora-CC-Text datasets, whereas the BoW features yield better results on the GoodBook-Text and PPI-Text datasets. This observation indicates that the BoW approach, despite its simplicity, is capable of capturing essential vertex feature information in certain contexts. However, when employing the proposed LM+NP method, we observe a substantial improvement in performance across all eleven datasets. Specifically, the introduction of neighborhood knowledge through fine-tuning enables the model to generate more informative and rich vertex embeddings, leading to enhanced accuracy in downstream tasks. These findings underscore the advantage of integrating neighborhood information into vertex feature representations. While both BoW and LM methods have their respective strengths, the LM+NP approach effectively combines the semantic richness of language models with neighborhood insights, resulting in more robust and accurate vertex embeddings. Consequently, the enhanced vertex knowledge embedded through our proposed method plays a crucial role in improving the overall performance of the Hypergraph Foundation Model on diverse Text-Attributed Hypergraph datasets.

2) *Ablation Study on Hypergraph Sampling Strategy*: The first step in constructing Hierarchical Multi-Hypergraphs involves structural sampling from multiple domain-specific hypergraph datasets. In this ablation study, we evaluate the impact of different sampling strategies on the performance of our proposed Hypergraph Foundation Model. Specifically, we compare our proposed BFS-based diffusion sampling strategy with a straightforward random vertex sampling strategy. The random vertex sampling strategy involves selecting vertices uniformly

TABLE V
ABLATION STUDY ON THE SAMPLING STRATEGY OF SUB-HYPERGRAPHS

	Random Sampling	BFS Sampling
Pubmed-CA-Text	0.4340 $\pm$.01	0.4603$\pm$.04
Movielens-Text	0.3168 $\pm$.09	0.3464$\pm$.08
GoodBook-Text	0.3351 $\pm$.02	0.3588$\pm$.04
PPI-Text	0.5048 $\pm$.09	0.5371$\pm$.08
Aminer-Text	0.4217 $\pm$.02	0.4325$\pm$.02
Cora-CA-Text	0.2462 $\pm$.02	0.2633$\pm$.03
Cora-CC-Text	0.3224 $\pm$.05	0.3706$\pm$.04
Pubmed-CC-Text	0.3902 $\pm$.01	0.4009$\pm$.01
Arxiv-Text	0.3960 $\pm$.05	0.4189$\pm$.04
IMDB-Text	0.2661 $\pm$.04	0.2676$\pm$.04
OGBN-Arxiv	0.2655 $\pm$.02	0.2789$\pm$.01

at random from the hypergraph and retaining the hyperedges that connect these selected vertices to form a sub-hypergraph. In contrast, the BFS-based sampling strategy systematically explores the hypergraph by expanding from an initial set of vertices, thereby preserving more of the inherent structural relationships within the hypergraph. The experimental results, as shown in Table V, indicate that the BFS-based sampling strategy significantly outperforms the random sampling strategy across all evaluated datasets. This superior performance can be attributed to the ability of BFS to maintain a greater extent of the hypergraph's associative structures compared to random sampling. Additionally, as the number of training epochs increases, the BFS strategy more effectively reconstructs the original hypergraph structural patterns, thereby minimizing the loss of structural information. Consequently, the BFS-based approach achieves a better balance between training speed and accuracy, demonstrating its efficacy in preserving essential hypergraph structures during the sampling process. These findings highlight the importance of the sampling strategy in the construction of Hierarchical Multi-Hypergraphs. By retaining more meaningful structural information, the BFS-based sampling approach enhances the model's ability to learn robust and generalizable representations, ultimately leading to improved performance on downstream tasks.

Furthermore, we conduct an ablation study to investigate the impact of the number of sampled vertices in the domain hypergraph sampling process. The experimental results are illustrated in Fig. 5. From the figure, it is evident that the performance across the datasets achieves its highest point when the number of sampled vertices is approximately 1000. When the number of sampled vertices is reduced to 500, the performance remains nearly unchanged. Considering both computational complexity and performance efficiency, we set the default vertex sampling count to 500. Additionally, our analysis reveals that sampling an excessively low or high number of vertices leads to a decline in performance. Specifically, sampling too few vertices results in a substantial loss of hypergraph structural knowledge, thereby degrading the model's performance. On the other hand, sampling too many vertices introduces increased structural redundancy within each domain. This redundancy does not introduce more domain structural patterns, thus restricting the effective transmission of information between domains and hampers the learning of general structural knowledge, ultimately

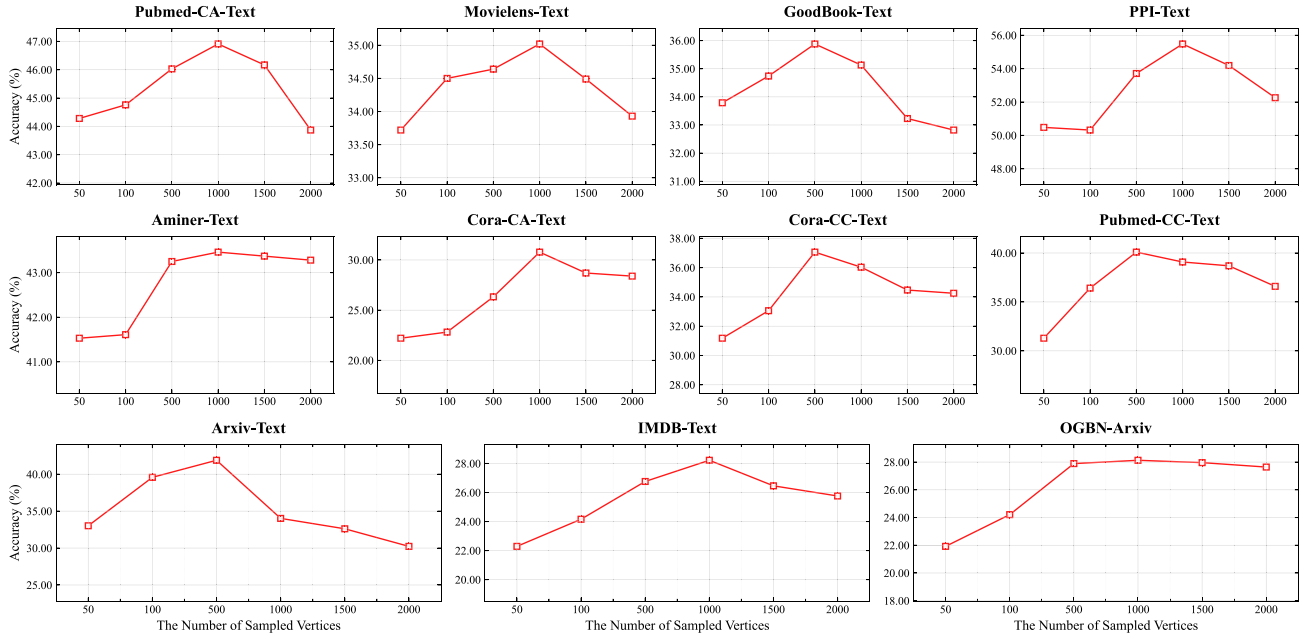


Fig. 5. Experimental results of ablation on the vertex number for the sub-hypergraph sampling for each hypergraph domain dataset.

TABLE VI
ABLATION STUDY ON VARYING THE NUMBER OF CLUSTERS IN THE HIERARCHICAL MULTI-HYPERGRAPH

	Number of Clustering						Number of Classes
	1	2	3	4	5	6	
Pubmed-CA-Text	0.4332 $\pm$.03	0.4460 $\pm$.03	0.4522 $\pm$.05	0.4465 $\pm$.01	0.4603$\pm$.04	0.4432 $\pm$.04	3
Movielens-Text	0.2884 $\pm$.02	0.3132 $\pm$.09	0.3281 $\pm$.07	0.3304 $\pm$.05	0.3464$\pm$.08	0.3217 $\pm$.07	5
GoodBook-Text	0.3378 $\pm$.06	0.3307 $\pm$.03	0.3294 $\pm$.03	0.3478 $\pm$.05	0.3588$\pm$.04	0.3227 $\pm$.04	4
PPI-Text	0.4919 $\pm$.09	0.5371 $\pm$.09	0.5581 $\pm$.09	0.5613$\pm$.09	0.5371 $\pm$.08	0.5016 $\pm$.06	4
Aminer-Text	0.4174 $\pm$.02	0.4195 $\pm$.02	0.4201 $\pm$.02	0.4284 $\pm$.02	0.4325$\pm$.02	0.4278 $\pm$.01	3
Cora-CA-Text	0.2535 $\pm$.05	0.2645 $\pm$.04	0.2637 $\pm$.04	0.2716 $\pm$.04	0.2633 $\pm$.03	0.2685$\pm$.03	7
Cora-CC-Text	0.3174 $\pm$.03	0.3431 $\pm$.04	0.3551 $\pm$.04	0.3595 $\pm$.05	0.3706 $\pm$.04	0.3735$\pm$.04	7
Pubmed-CC-Text	0.3531 $\pm$.06	0.3577 $\pm$.06	0.3856 $\pm$.01	0.3918 $\pm$.01	0.4009$\pm$.01	0.3948 $\pm$.01	3
Arxiv-Text	0.3345 $\pm$.05	0.3315 $\pm$.05	0.3198 $\pm$.04	0.3329 $\pm$.08	0.4189$\pm$.04	0.3270 $\pm$.08	5
IMDB-Text	0.2417 $\pm$.01	0.2452 $\pm$.01	0.2520 $\pm$.04	0.2820 $\pm$.05	0.2676 $\pm$.04	0.2748$\pm$.04	5
OGBN-Arxiv	0.2337 $\pm$.04	0.2401 $\pm$.04	0.2637 $\pm$.04	0.2728 $\pm$.02	0.2789$\pm$.01	0.2756 $\pm$.04	40

diminishing the utility of the pre-trained model. These findings highlight the importance of selecting an optimal number of sampled vertices to balance the preservation of structural information and the efficiency of knowledge transfer across domains. By setting the vertex sampling count to 500, we achieve a favorable trade-off between maintaining essential hypergraph structures and ensuring computational feasibility, thereby enhancing the overall performance of the Hypergraph Foundation Model.

3) *Ablation Study on Structure Knowledge Extraction*: In our Hyper-FM, the core of structural knowledge extraction lies in the construction of hierarchical multi-hypergraphs. The main experiments have already demonstrated that jointly integrating multi-domain hypergraphs can significantly enhance the model's ability to extract structural knowledge from hypergraphs. In this ablation study, we further investigate the impact of the connection structure within the jointly integrated multi-domain hypergraphs. Specifically, we evaluate two different configurations: (1) without using clustering, directly connecting

vertices from corresponding domain-specific hypergraphs (resulting in a non-hierarchical hypergraph), (2) using clustering to construct hierarchical hypergraphs and varying the number of clusters to observe its effect on the final performance. The experimental results are presented in Table VI. When constructing multi-domain hypergraphs without clustering—indicated by a clustering number of 1 in the table—versus using clustering to build hierarchical hypergraphs, we can make two key observations.

Firstly, employing clustering to construct hierarchical hypergraphs leads to a significant performance improvement. This enhancement is attributable to the inherent complexity of hypergraph structures and the natural formation of distinct semantic clusters within the same domain due to varying vertex labels. Directly connecting multi-domain datasets that contain different semantic meanings without clustering impedes the effective transmission of information across the hypergraph, resulting in a decline in performance. Secondly, we observe that the number of clusters used in the clustering process has a varying impact on

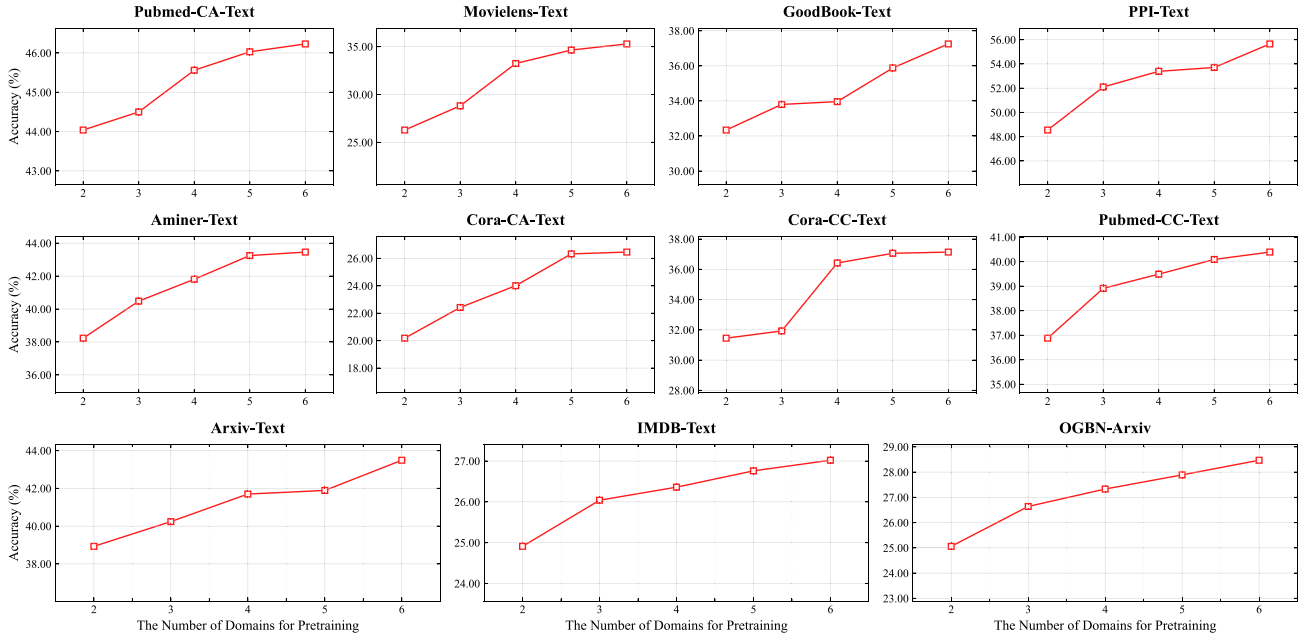


Fig. 6. Experimental results of ablation on the number of domains for pretraining.

the capability of the Hypergraph Foundation Model. The optimal number of clusters in the hierarchical hypergraph is found to be approximately equal to the number of classes in the dataset. This observation indirectly suggests that clustering based on vertex semantic information effectively reflects the distribution of vertex labels. Consequently, the setting of cluster numbers can be guided by the number of classes present in the original data, providing a practical heuristic for setting this hyperparameter. These findings underscore the critical role of clustering in constructing hierarchical multi-hypergraphs, as it facilitates the preservation and effective utilization of semantic and structural information within and across domains. By aligning the number of clusters with the underlying class structure of the data, the model can better capture and leverage the nuanced relationships inherent in hypergraph-structured datasets, leading to enhanced performance in downstream tasks.

E. Scaling Law of Hypergraph Foundation Model

In most foundation models, the model's capability increases with the amount of data, as observed in CV [25] and NLP models [15]. However, relational data differ in their representation from these domains. Unlike CV data, which can be directly visualized, or NLP data, which can be directly interpreted, relational data primarily model the connections within real-world networks. As illustrated in Fig. 5, the performance of foundation models does not necessarily improve with an increase in the volume of relational data. In this subsection, we explore the scaling law of the Hypergraph Foundation Model from a different perspective—the number of domains. The experimental results are presented in Fig. 6. Each domain-specific hypergraph dataset possesses unique connection patterns, and an increase in the number of domains introduces a variety of relational data patterns. This diversity injects more comprehensive knowledge

into the Hypergraph Foundation Model. Remarkably, the experimental findings reveal that as the number of domains increases, the performance of the Hypergraph Foundation Model on downstream tasks continues to improve. This trend suggests that incorporating a greater variety of relational structures, each with its distinct patterns, enhances the model's ability to learn and generalize. **The addition of diverse domain-specific knowledge contributes more effectively to the foundation model's power than merely increasing the number of vertices and hyperedges within the hypergraphs.** These results highlight that the richness and diversity of relational structures across multiple domains play a crucial role in scaling the capabilities of hypergraph-based foundation models. By leveraging information from varied connection patterns, the Hypergraph Foundation Model can better capture complex relationships and dependencies, leading to superior performance in downstream applications. This finding underscores the importance of multi-domain integration in the development and scaling of foundation models for relational data.

VI. CONCLUSION

In this work, we introduce Hyper-FM, the first Hypergraph Foundation Model for multi-domain knowledge extraction. By implementing Hierarchical High-Order Neighbor Guided Vertex Knowledge Embedding, we integrate domain-specific information into vertex features, addressing varying dimensions and structural complexities. Our Hierarchical Multi-Hypergraph Guided Structural Knowledge Extraction method allows scalable and efficient extraction of structural knowledge from diverse datasets, mitigating negative transfer and domain-specific blending. Additionally, we curate 11 text-attributed hypergraph (TAHG) datasets to bridge hypergraph neural networks and large language models. Empirical evaluations show that Hyper-FM

outperforms baseline methods by approximately 10% across these datasets, demonstrating its robustness and effectiveness. We also establish a novel scaling law, revealing that increasing domain diversity significantly enhances model performance, unlike merely expanding vertex and hyperedge counts. Hyper-FM sets a new benchmark for hypergraph foundation models, providing a scalable and effective solution for multi-domain knowledge extraction. Future work will explore expanding TAHG datasets and applying Hyper-FM to a broader range of domains to further enhance its versatility and impact.

REFERENCES

- [1] K. A. Murgas, E. Saucan, and R. Sandhu, "Hypergraph geometry reflects higher-order dynamics in protein interaction networks," *Sci. Rep.*, vol. 12, no. 1, 2022, Art. no. 20879.
- [2] Y. Jiang et al., "Explainable deep hypergraph learning modeling the peptide secondary structure prediction," *Adv. Sci.*, vol. 10, no. 11, 2023, Art. no. 2206151.
- [3] J. Wang et al., "Dynamic weighted hypergraph convolutional network for brain functional connectome analysis," *Med. Image Anal.*, vol. 87, 2023, Art. no. 102828.
- [4] X. Han, R. Xue, S. Du, and Y. Gao, "Inter-intra high-order brain network for ASD diagnosis via functional MRIs," in *Proc. Int. Conf. Med. Image Comput. Comput. Assist. Intervention*, 2024, pp. 216–226.
- [5] C. Bick, E. Gross, H. A. Harrington, and M. T. Schaub, "What are higher-order networks?," *SIAM Rev.*, vol. 65, no. 3, pp. 686–731, 2023.
- [6] Q. F. Lotito, F. Musciotto, A. Montresor, and F. Battiston, "Hyperlink communities in higher-order networks," *J. Complex Netw.*, vol. 12, no. 2, 2024, Art. no. cnae013.
- [7] R. D. Hjelm et al., "Learning deep representations by mutual information estimation and maximization," in *Proc. Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bklr3j0cKX>
- [8] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [9] H. Fang, S. Wang, M. Zhou, J. Ding, and P. Xie, "CERT: Contrastive self-supervised learning for language understanding," 2020, *arXiv:2005.12766*.
- [10] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 6894–6910. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.552/>
- [11] S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in vision: A survey," *ACM Comput. Surv.*, vol. 54, no. 10s, pp. 1–41, 2022.
- [12] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, vol. 1, 2019, pp. 4171–4186.
- [13] Z. Liu, W. Lin, Y. Shi, and J. Zhao, "A robustly optimized BERT pretraining approach with Post-training," in *Proc. 20th Chin. Nat. Conf. Comput. Linguistics*, Huhhot, China, Aug. 2021, pp. 1218–1227.
- [14] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, 2019, Art. no. 9.
- [15] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [16] Y. Zhang, R. Jin, and Z.-H. Zhou, "Understanding bag-of-words model: A statistical framework," *Int. J. Mach. Learn. Cybern.*, vol. 1, pp. 43–52, 2010.
- [17] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman, "Indexing by latent semantic analysis," *J. Amer. Soc. Inf. Sci.*, vol. 41, no. 6, pp. 391–407, 1990.
- [18] T. Wei, Y. You, T. Chen, Y. Shen, J. He, and Z. Wang, "Augmentations in hypergraph contrastive learning: Fabricated and generative," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 1909–1922.
- [19] D. Lee and K. Shin, "I'm me, we're us, and I'm us: Tri-directional contrastive learning on hypergraphs," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 8456–8464.
- [20] S. Kim, S. Kang, F. Bu, S. Y. Lee, J. Yoo, and K. Shin, "HypeBoy: Generative self-supervised representation learning on hypergraphs," in *Proc. 12th Int. Conf. Learn. Representations*, 2024.
- [21] J. Zhao et al., "Learning on large-scale text-attributed graphs via variational inference," in *Proc. 11th Int. Conf. Learn. Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=q0nmYciuuZN>
- [22] H. Yan et al., "A comprehensive study on text-attributed graphs: Benchmarking and rethinking," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 17238–17264.
- [23] X. He, X. Bresson, T. Laurent, A. Perold, Y. LeCun, and B. Hooi, "Harnessing explanations: LLM-to-LM interpreter for enhanced text-attributed graph representation learning," in *Proc. 12th Int. Conf. Learn. Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=RXFVcynVei>
- [24] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.
- [25] M. Caron et al., "Emerging properties in self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 9650–9660.
- [26] A. Kirillov et al., "Segment anything," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 4015–4026.
- [27] X. Wang, X. Zhang, Y. Cao, W. Wang, C. Shen, and T. Huang, "SegGPT: Towards segmenting everything in context," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 1130–1140.
- [28] C. Wu, S. Yin, W. Qi, X. Wang, Z. Tang, and N. Duan, "Visual ChatGPT: Talking, drawing and editing with visual foundation models," 2023, *arXiv:2303.04671*.
- [29] R. Ye et al., "Language is all a graph needs," *Findings Assoc. Comput. Linguistics: EACL*, pp. 1955–1973, 2024.
- [30] J. Tang et al., "GraphGPT: Graph instruction tuning for large language models," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2024, pp. 491–500.
- [31] R. Chen, T. Zhao, A. Jaiswal, N. Shah, and Z. Wang, "LLaGA: Large language and graph assistant," in *Proc. Forty-first Int. Conf. Mach. Learn.*, 2024. [Online]. Available: <https://openreview.net/forum?id=B48Pzc4oKi>
- [32] H. Luo et al., "Enhance graph alignment for large language models," 2024, *arXiv:2410.11370*.
- [33] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 3558–3565.
- [34] Y. Gao, Y. Feng, S. Ji, and R. Ji, "HGNN: General hypergraph neural networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3181–3199, Mar. 2023.
- [35] N. Yadati, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar, "HyperGCN: A new method for training graph convolutional networks on hypergraphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 1511–1522.
- [36] Y. Dong, W. Sawin, and Y. Bengio, "HNHN: Hypergraph networks with hyperedge neurons," in *Proc. Int. Conf. Mach. Learn.*, 2020.
- [37] S. Bai, F. Zhang, and P. H. Torr, "Hypergraph convolution and hypergraph attention," *Pattern Recognit.*, vol. 110, 2021, Art. no. 107637.
- [38] J. Huang and J. Yang, "UniGNN: A unified framework for graph and hypergraph neural networks," in *Proc. 13th Int. Joint Conf. Artif. Intell.*, International Joint Conferences on Artificial Intelligence Organization, 2021, pp. 2563–2569.
- [39] R. Zhang, Y. Zou, and J. Ma, "HyperSAGE: Generalizing inductive representation learning on hypergraphs," in *Proc. Int. Conf. Learn. Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=ryeHuJBtPH>
- [40] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [41] E. Chien, C. Pan, J. Peng, and O. Milenkovic, "You are AllSet: A multitask function framework for hypergraph neural networks," in *Proc. Int. Conf. Learn. Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=hpBTiv2uy_E
- [42] Z. Mei, X. Bi, D. Li, W. Xia, F. Yang, and H. Wu, "DHHNN: A dynamic hypergraph hyperbolic neural network based on variational autoencoder for multimodal data integration and node classification," *Inf. Fusion*, vol. 119, 2025, Art. no. 103016.
- [43] X. Han et al., "Hypergraph foundation model for brain disease diagnosis," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 10, pp. 17702–17716, Oct. 2025.
- [44] X. Huang, M. Galkin, M. M. Bronstein, and İ. İ. Ceylan, "HYPER: A foundation model for inductive link prediction with knowledge hypergraphs," *New Perspectives Graph Mach. Learn.*, 2025. [Online]. Available: <https://openreview.net/forum?id=X1fKViBAIU>

- [45] X. Zhou et al., "Dual-masked contrastive learning based hypergraph foundation model for whole slide images," *Pattern Recognit.*, vol. 169, 2025, Art. no. 111995.
- [46] S. Dong, D. Li, Y. Sun, X. Hu, and Y. Zhao, "Hypergraph-driven landmark detection foundation model on echocardiography for cardiac function quantification," *Pattern Recognit.*, vol. 170, 2025, Art. no. 111927.
- [47] J. Zhang, W.-C. Chang, H.-F. Yu, and I. Dhillon, "Fast multi-resolution transformer fine-tuning for extreme multi-label text classification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 7267–7280.
- [48] Y. Prabhu, A. Kag, S. Harsola, R. Agrawal, and M. Varma, "Parabel: Partitioned label trees for extreme classification with application to dynamic search advertising," in *Proc. World Wide Web Conf.*, 2018, pp. 993–1002.
- [49] S. Khandagale, H. Xiao, and R. Babbar, "Bonsai: Diverse and shallow trees for extreme multi-label classification," *Mach. Learn.*, vol. 109, no. 11, pp. 2099–2119, 2020.
- [50] L.-P. Jing, H.-K. Huang, and H.-B. Shi, "Improved feature selection approach TFIDF in text mining," in *Proc. Int. Conf. Mach. Learn. Cybern.*, 2002, pp. 944–946.
- [51] Y. Feng, S. Liu, S. Ying, S. Du, Z. Wu, and Y. Gao, "Self-supervised hypergraph training framework via structure-aware learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 11, pp. 10160–10176, Nov. 2025.
- [52] Z. Hou et al., "GraphMAE: Self-supervised masked graph autoencoders," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 594–604.
- [53] H. Zhao, A. Chen, X. Sun, H. Cheng, and J. Li, "All in one and one for all: A simple yet effective method towards cross-domain graph pretraining," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 4443–4454.
- [54] K. Wang, Z. Shen, C. Huang, C.-H. Wu, Y. Dong, and A. Kanakia, "Microsoft academic graph: When experts are not enough," *Quantitative Sci. Stud.*, vol. 1, no. 1, pp. 396–413, 2020.
- [55] E. Chien et al., "Node feature extraction by self-supervised multi-scale neighborhood prediction," in *Proc. Int. Conf. Learn. Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=KJgglHbs8>



Yue Gao (Senior Member, IEEE) received the BS degree from the Harbin Institute of Technology, Harbin, China, and the ME and PhD degrees from Tsinghua University, Beijing, China. He is currently an associate professor with the School of Software, Tsinghua University.



Yifan Feng received the BE degree in computer science and technology from Xidian University, Xi'an, China, in 2018, the MS degree from Xiamen University, Xiamen, China, in 2021, and the PhD degree from Tsinghua University, Beijing, China, in 2025. His research interests include hypergraph learning, multi-modal learning, and large language model.



Shiquan Liu received the BE degree from Xi'an Jiaotong University, Xi'an, China, in 2023. He is currently working toward the master's degree in artificial intelligence from Xi'an Jiaotong University.



Xiangmin Han (Member, IEEE) is currently a post-doctoral with the School of Software, Tsinghua University. His research interests include medical image analysis, brain network analysis, and hypergraph computation.



Shaoyi Du received the dual bachelor's degrees in computational mathematics and in computer science in 2002 and the MS degree in applied mathematics in 2005, and the PhD degree in pattern recognition and intelligence system from Xi'an Jiaotong University, China in 2009. He is currently a professor with Xi'an Jiaotong University. His research interests include computer vision, machine learning, and pattern recognition.



Zongze Wu (Member, IEEE) received the BS degree in material forming and control, the MS degree in control science and engineering, and the PhD degree in pattern reorganization and intelligence system from Xi'an Jiaotong University, Xi'an, China, in 1999, 2002, and 2005, respectively. He is currently a professor with the College of Mechatronics and Control Engineering, Shenzhen University, China, where he is also with the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ). He is the author and coauthor of 70 SCI indexed papers, which include *IEEE Transactions on Automatic Control*, *IEEE Transactions on Cybernetics*, *IEEE Systems, Man, and Cybernetics: Systems*, *IEEE/CAA Journal of Automatica Sinica*, *IEEE Transactions on Image Processing*, *Pattern Recognition*, and *Signal Processing*.



Han Hu (Member, IEEE) received the BE and PhD degrees from the University of Science and Technology of China, China, in 2007 and 2012, respectively. He is currently a professor with the School of Information and Electronics, Beijing Institute of Technology, China. His research interests include multimedia networking, edge intelligence, and space-air-ground integrated network. He was the recipient of the several academic awards, including the Best Paper Award of the IEEE TCSVT 2019, the Best Paper Award of the IEEE Multimedia Magazine 2015, and the Best Paper Award of the IEEE Globecom 2013. He was an associate editor for *IEEE Transactions on Multimedia* and *Ad Hoc Networks*, and a TPC member of Infocom, ACM MM, AAAI, and IJCAI.